Oracle 19c Real Application Clusters

# RAC Administration

RAC Administration and Deployment Guide 19c · Clusterware Administration Guide · 5일 과정 10~14 · 17장

## 운영과 관리

# 이 자료로 설명할 수 있어야 하는 것

- 공유하는 파일과 인스턴스별로 나누는 파일을 가른다
- srvctl 과 crsctl 중 맞는 명령을 고르고 오류 줄을 읽는다
- 서비스로 접속을 나누고 옮기며, relocate 와 장애의 차이를 안다
- FAN · TAF · AC 가 각각 무엇을 지키는지 말한다
- RAC 에서 아카이브 · RMAN 채널 · 복구를 구성한다
- gc 대기와 로그로 성능 문제와 장애를 좁혀 간다

# Agenda

- 1 RAC 스토리지 관리
- 2 srvctl 과 crsctl
- 3 동적 데이터베이스 서비스
- 4 FAN · TAF · Application Continuity
- 5 RAC 백업과 복구
- 6 진단 · 성능 · 종합 운영

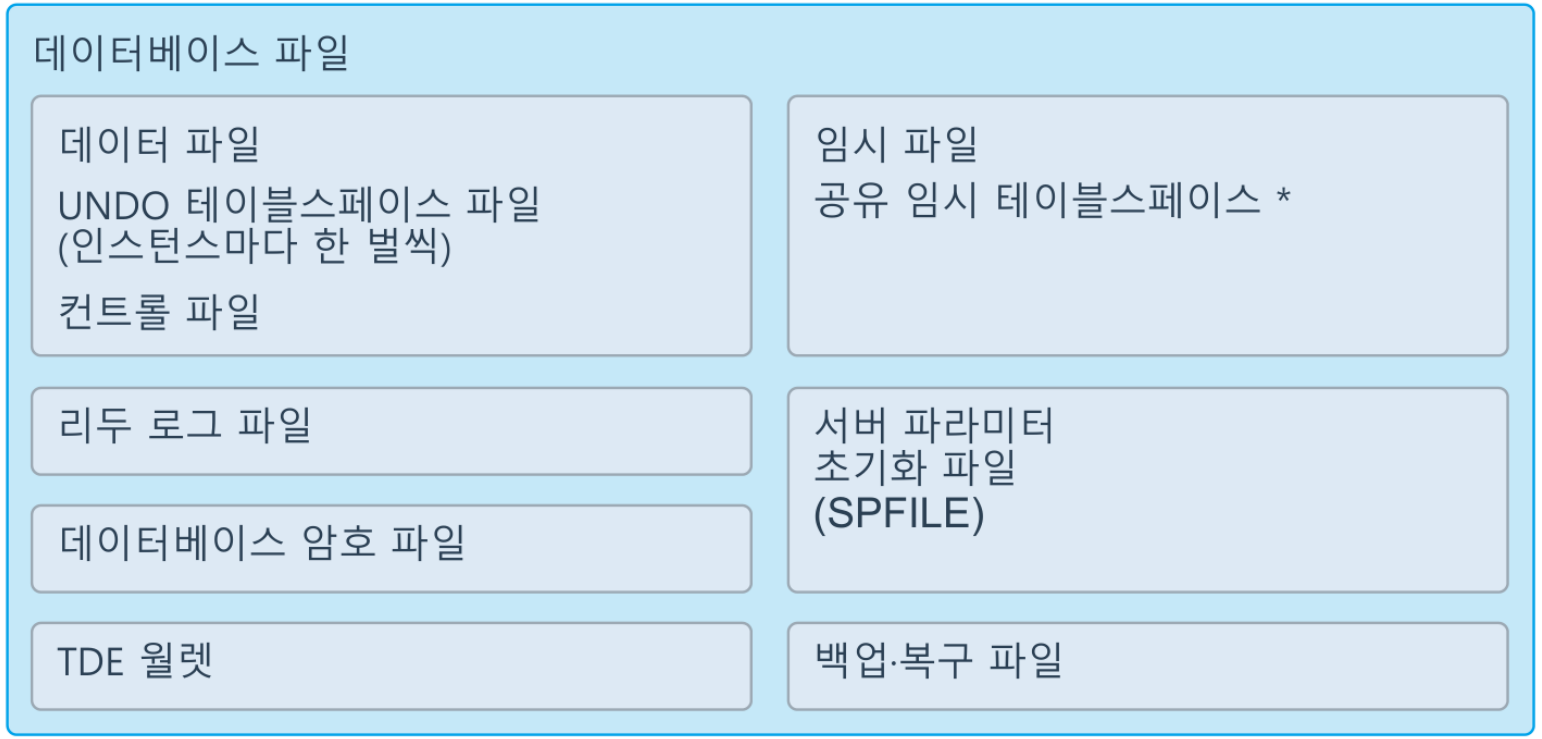
# 1

# RAC 스토리지 관리

- 공유하는 것과 나누는 것
- 리두 스레드와 그룹 관리
- UNDO · 임시 테이블스페이스
- 인스턴스별 파라미터와 노드 추가 준비

# RAC 데이터베이스를 이루는 파일

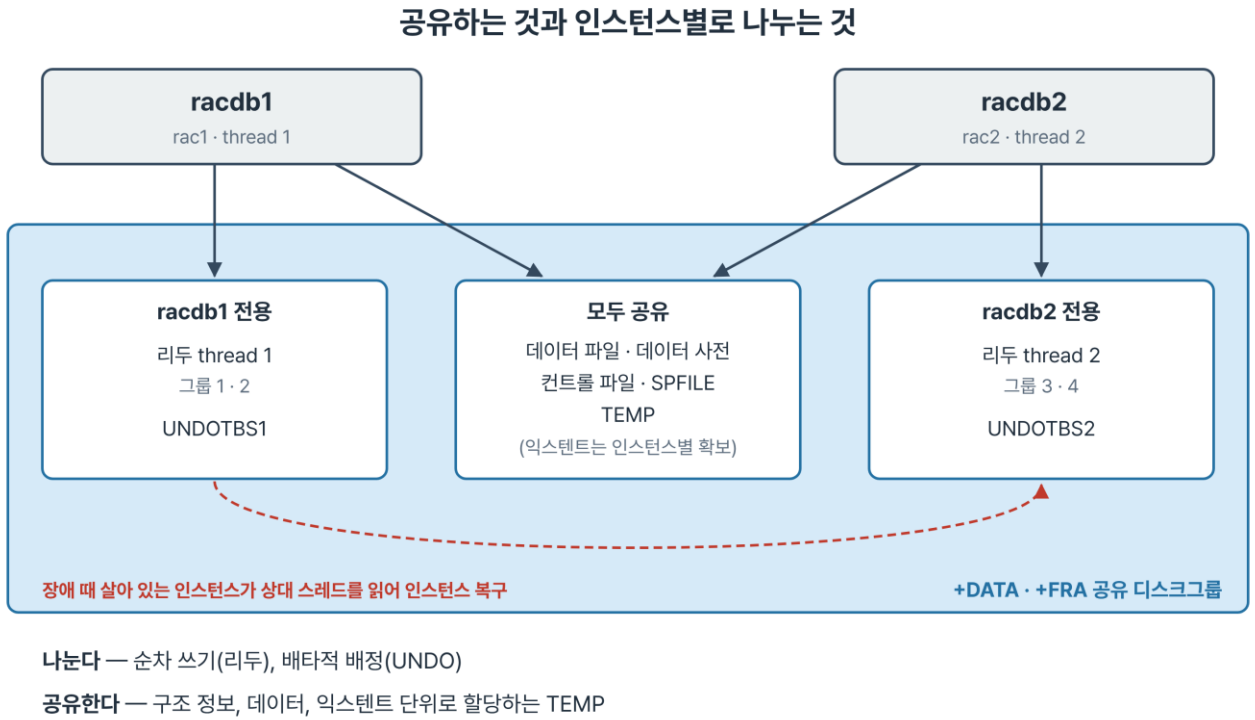
모든 파일은 공유 스토리지에 (원문 11쪽)




- 데이터 파일 · 컨트롤 파일 · SPFILE · TEMP — 공유
- 리두 스레드 · UNDO — 인스턴스마다 하나씩
- 리두는 인스턴스마다 최소 두 그룹
- raw 장치는 Clusterware 12c 부터 지원 안 함

# 공유하는 것과 나누는 것

기준은 하나 — 동시에 써도 되는가




- 리두: 순차 쓰기 → 나눈다
- UNDO: 롤백 세그먼트 배타 배정 → 나눈다
- TEMP: 익스텐트 단위 할당 → 공유
- 나뉜 자원도 파일은 공유 디스크그룹에

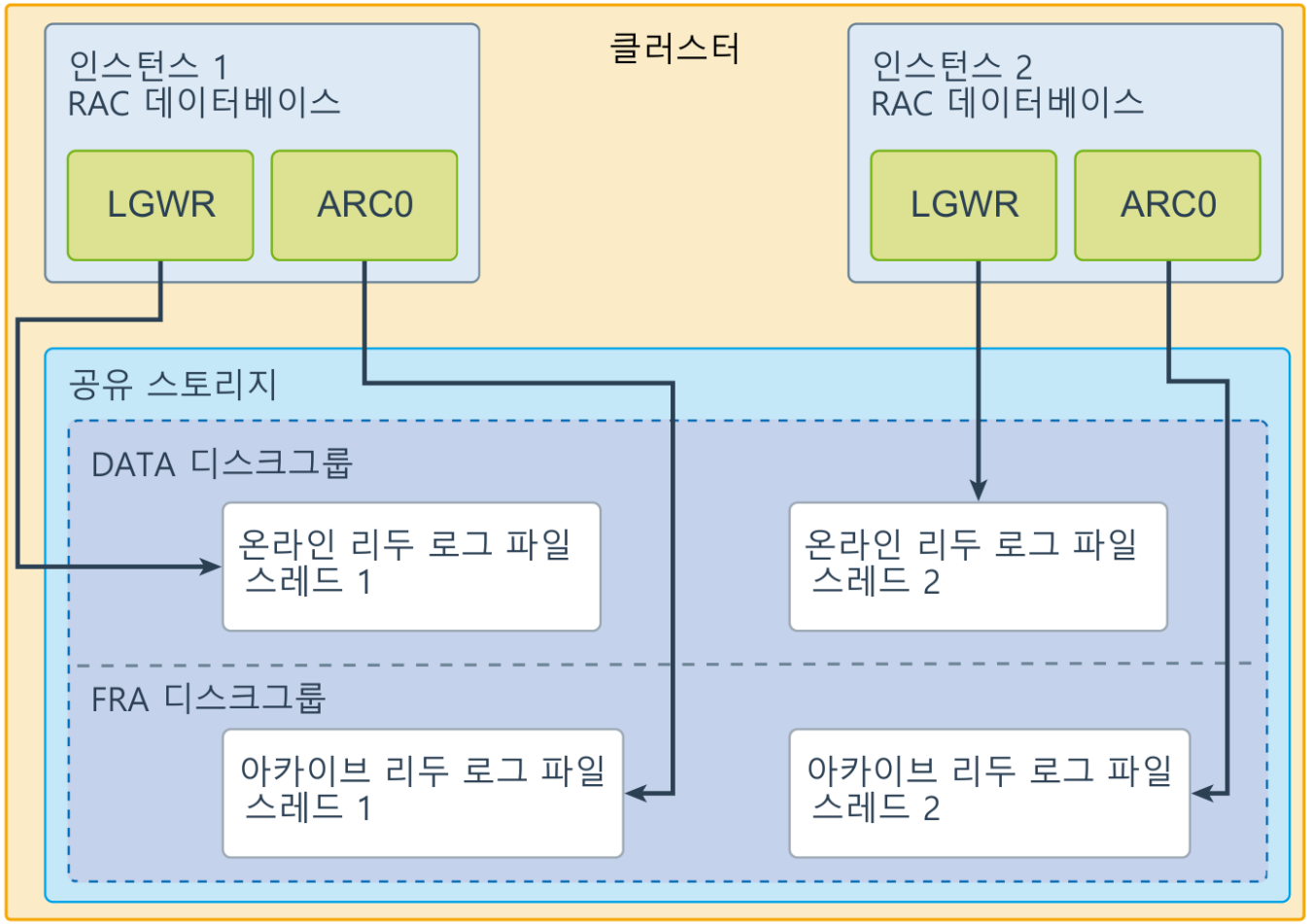
# 데이터 파일 접근

모든 인스턴스가 모든 파일에 닿는가

- 열 때 · 파일 추가 때 · ONLINE 때 인스턴스마다 확인
- 실패하면 원인 수정 후 ALTER SYSTEM CHECK DATAFILES
- 실측: rac2 에서 만든 TS_CH13 이 rac1 에 바로 보임
- 양쪽에서 1000 건씩 넣고 2000 건 확인
- 로컬 경로 파일은 조회는 되고 쓰기에서 ORA-01157

# 리두 스레드

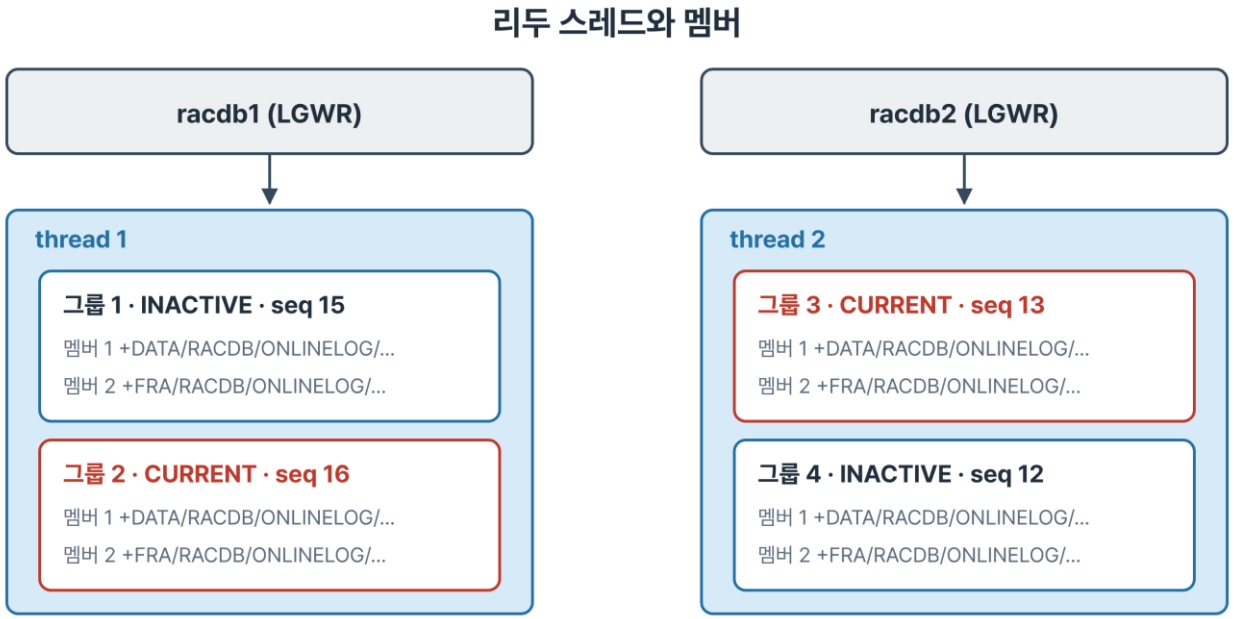
인스턴스마다 자기 스레드에 쓰고 아카이브한다 (원문 16~17쪽)




- 스레드 = 한 인스턴스의 리두 그룹 묶음
- 스레드마다 CURRENT 하나, SEQUENCE# 도 따로
- 복구는 여러 스레드를 SCN 순서로 병합
- THREAD 파라미터는 인스턴스마다 달라야 한다

# 스레드와 멤버

누가 쓰는가와 몇 벌 두는가는 다른 축




**스레드** = 누가 쓰는가 (RAC 에서 생긴 축) · 스레드마다 CURRENT 하나, SEQUENCE# 도 따로

**멤버** = 몇 벌 두는가 (다중화, 단일 인스턴스에도 있음)

4 그룹 × 2 멤버 → V$LOGFILE 조인 결과 8행

- 그룹 4 × 멤버 2 → V$LOGFILE 조인 8행
- 멤버는 +DATA · +FRA 에 하나씩
- 스레드 1 은 racdb1 만 쓰지만 racdb2 도 읽을 수 있다

# 리두 그룹 추가

THREAD 절을 빠뜨려도 오류가 없다

- add logfile thread 1 group 5 ('+DATA','+FRA') size 50M — 3.2초
- rac2 에서 즉시 보임, 쓰는 것은 racdb1
- 새 그룹 STATUS = UNUSED
- 문서는 INSTANCE 절, 실습은 THREAD 절
- 절이 없으면 접속한 인스턴스의 스레드에 붙는다

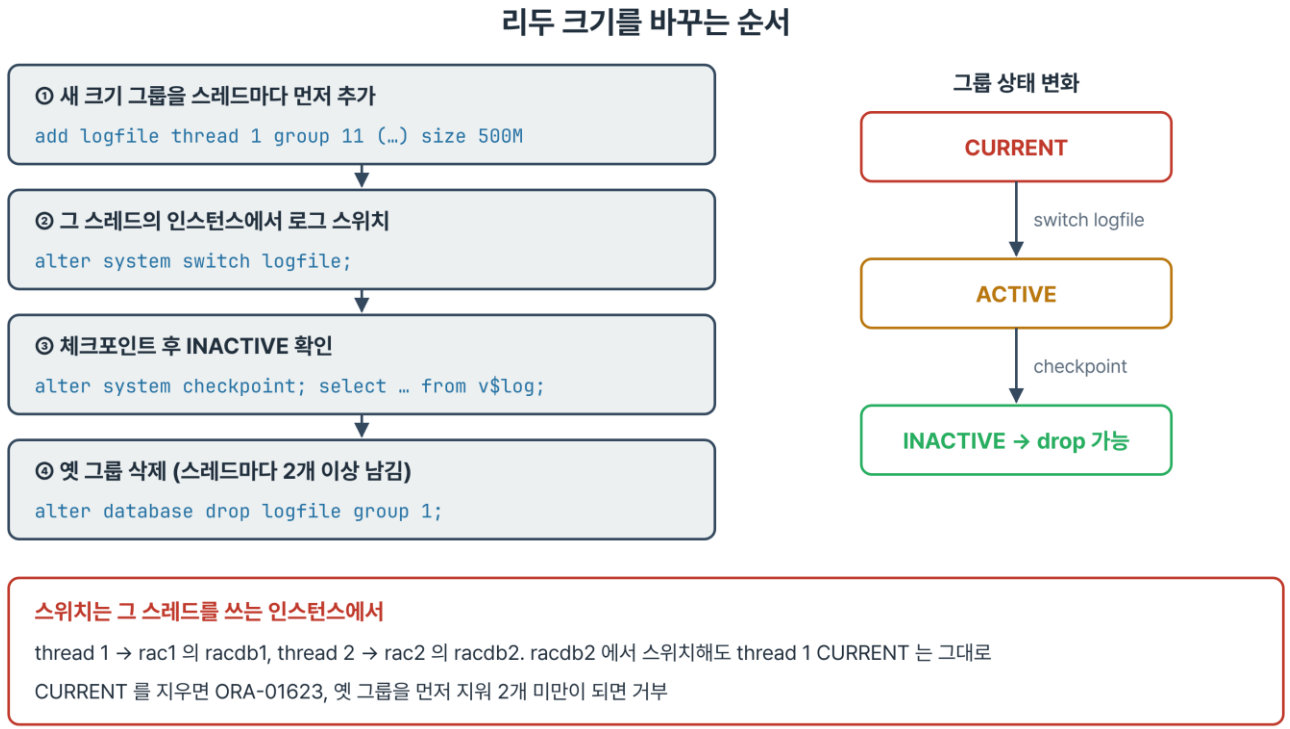
# ORA-01623

지금 쓰는 리두는 지울 수 없다

- log 2 is current log for instance racdb1 (thread 1)
- 메시지가 스위치할 노드를 알려 준다
- ORA-00312 두 줄 = 멤버 두 개
- racdb2 에서 스위치해도 스레드 1 은 그대로

# 리두 크기 변경

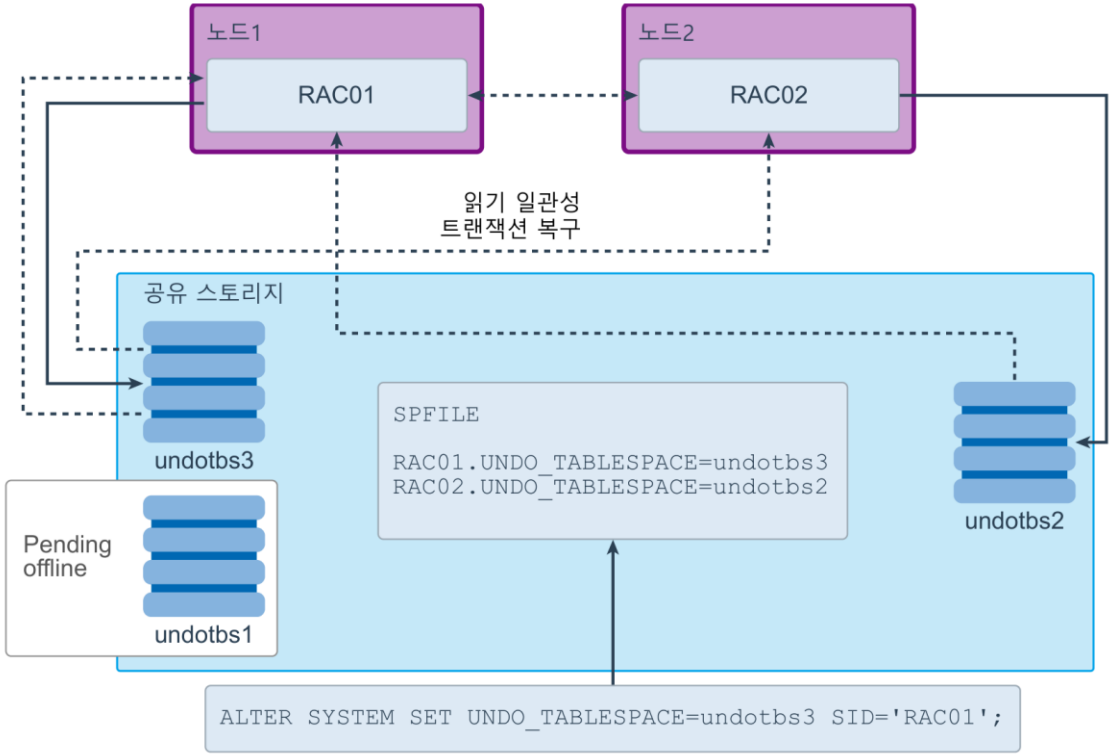
새로 만들고 옛것을 뺀다




- 1 새 크기 그룹을 스레드마다 먼저 추가
- 2 스레드 1 은 rac1, 스레드 2 는 rac2 에서 스위치
- 3 INACTIVE 확인 후 옛 그룹 삭제
- 스레드마다 크기를 같게

# UNDO 테이블스페이스

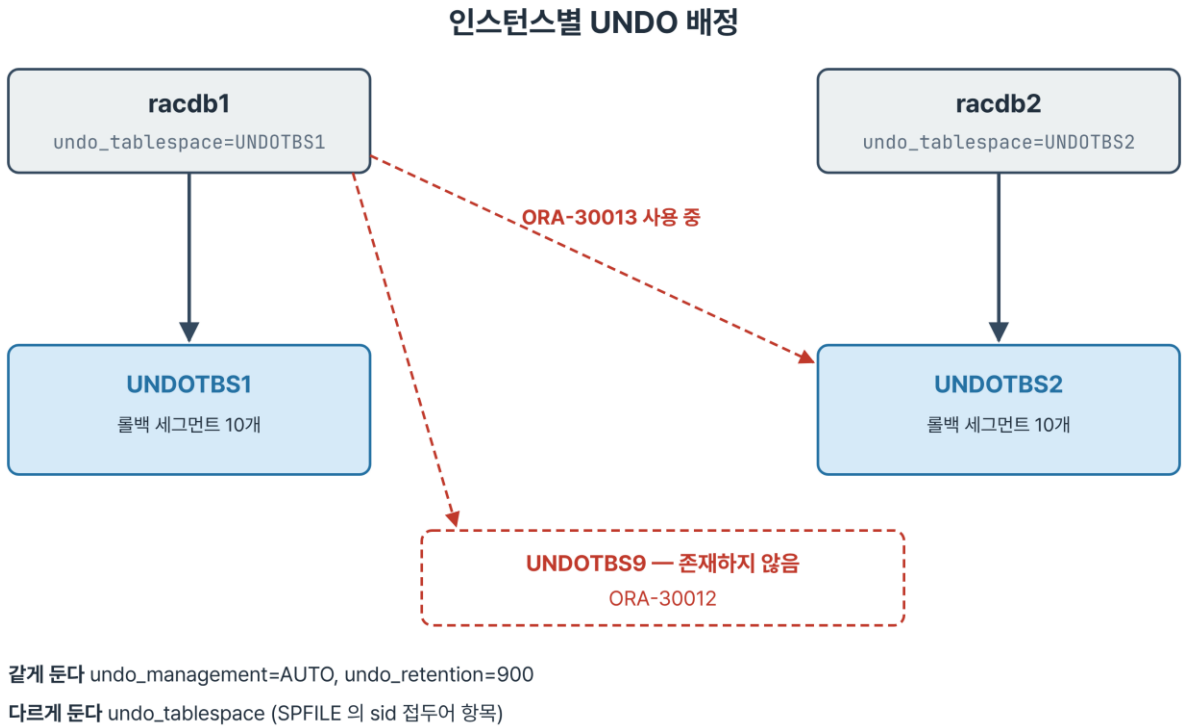
배정받은 인스턴스만 바꾸고 모두가 읽는다 (원문 15쪽)




- UNDO_TABLESPACE 는 인스턴스마다 다르게
- 읽기 일관성용 읽기는 어느 인스턴스든
- 트랜잭션 복구 중에는 남의 UNDO 도 갱신
- 자동 · 수동 UNDO 관리를 섞을 수 없다

# 인스턴스별 UNDO 배정

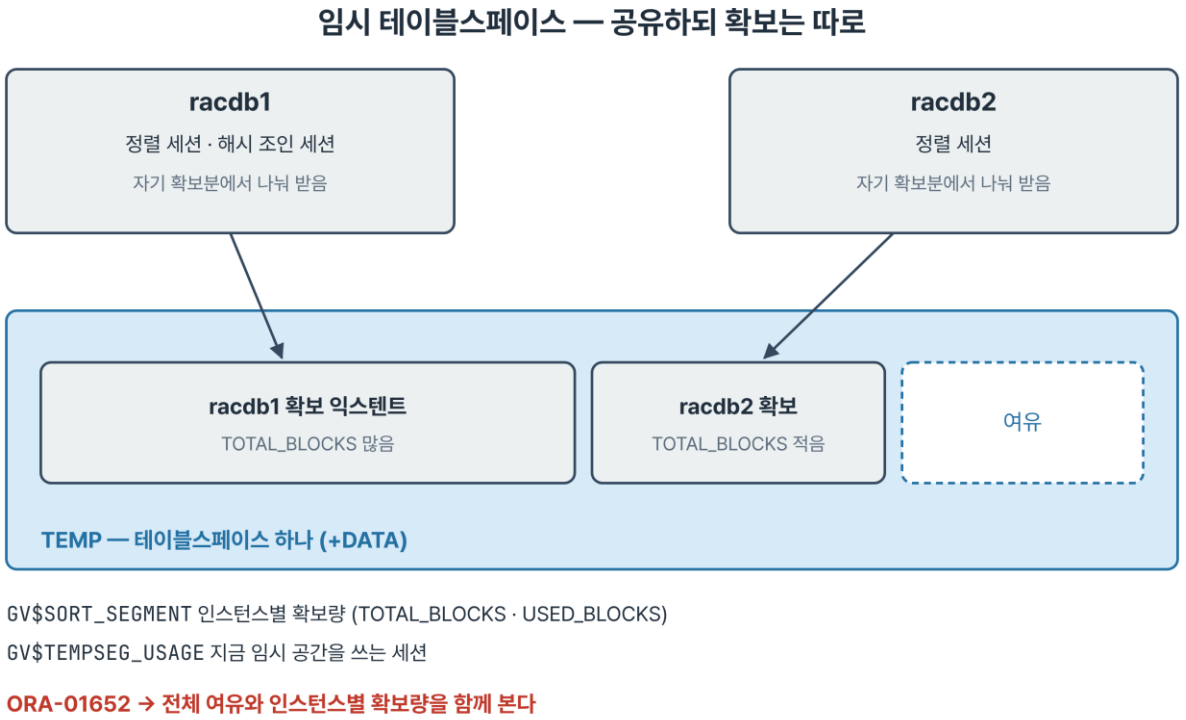
남의 것은 ORA-30013, 없는 것은 ORA-30012




- 실측: UNDOTBS1 · UNDOTBS2, 롤백 세그먼트 10개씩
- undo_retention 900 은 같게 — 문서도 같게 권장
- 통계는 GV$UNDOSTAT — MAXQUERYLEN 1561초 구간
- scope=memory 로 시험하면 SPFILE 에 흔적이 없다

# 임시 테이블스페이스

하나를 공유하고 확보는 인스턴스별로




- TEMP 는 하나, 임시 파일 셋은 컨테이너 셋
- 인스턴스 1 은 2688+512, 2 는 1280+384 블록 확보
- ORA-01652: DBA_TEMP_FILES 와 GV$SORT_SEGMENT 를 함께
- 12.2+ LOCAL TEMPORARY TABLESPACE 선택지

# 같게 · 다르게

스토리지에 걸린 초기화 파라미터




- 반드시 같게: CONTROL_FILES · DB_RECOVERY_FILE_DEST · UNDO_MANAGEMENT
- 반드시 다르게: INSTANCE_NUMBER · THREAD · UNDO_TABLESPACE
- 같게 권장: UNDO_RETENTION · LOG_ARCHIVE_FORMAT
- SPFILE 은 하나, 인스턴스 값은 sid 접두어

# 인스턴스를 늘리고 줄일 때

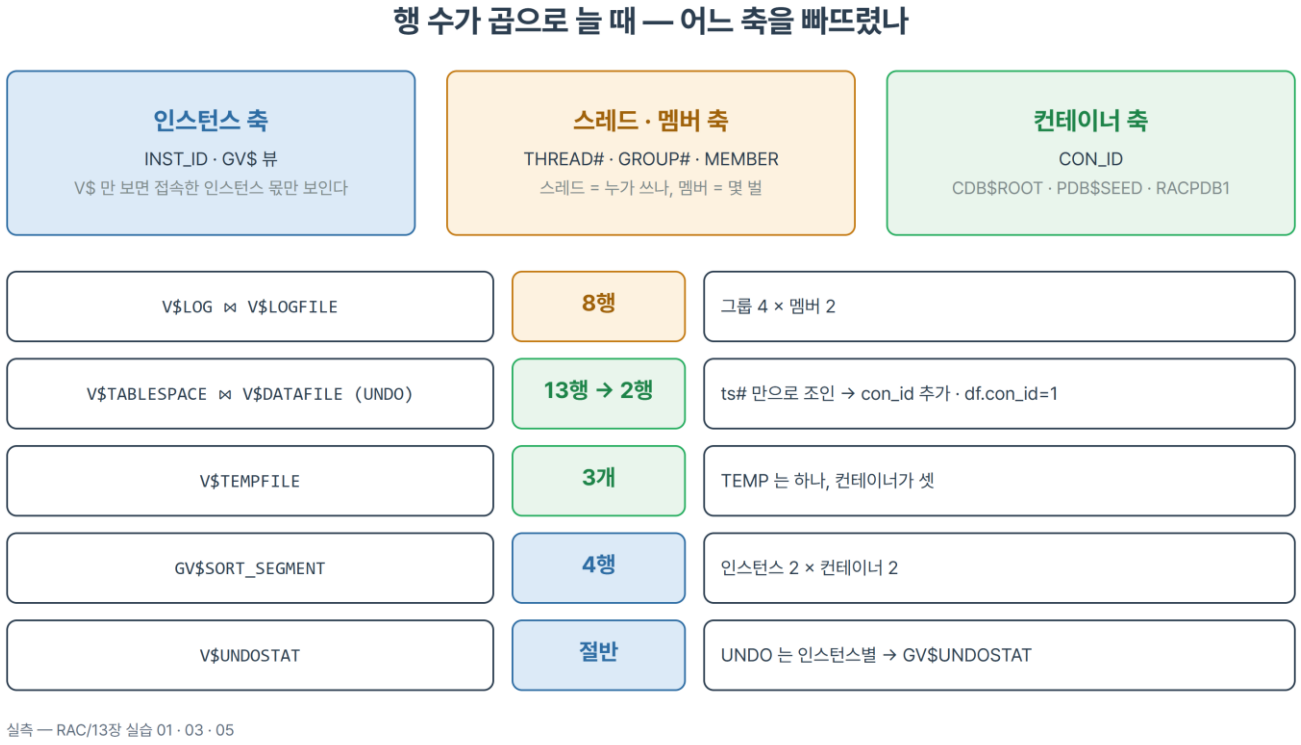
리두 그룹 → 스레드 → UNDO → SPFILE




- 그룹 둘 먼저, 그다음 ENABLE THREAD (문서: ENABLE INSTANCE)
- UNDO 를 만들지 않으면 30012, 기존 것이면 30013
- TEMP · 데이터 파일 · 컨트롤 파일은 그대로
- 뺄 때는 DISABLE THREAD — 복구가 enabled 스레드를 모두 검사

# 조회의 함정

행 수가 곱으로 늘면 축을 빠뜨린 것




- 인스턴스 축: GV$ 를 쓰지 않으면 절반
- 스레드 · 멤버 축: 8행 = 4 × 2
- 컨테이너 축: UNDO 조인 13행 → con_id 넣고 2행
- GV$ROLLNAME 은 없다 — ORA-00942

# 부하 편중을 수치로

GV$SYSSTAT · V$LOG_HISTORY

- 스위치 이력: 스레드 1 15회, 스레드 2 12회
- redo size: rac1 이 약 1.8배, redo writes 도 같은 비율
- redo log space requests 0 — 그룹 크기는 충분
- 조치는 리두 · UNDO 확대가 아니라 접속 분산(3장)

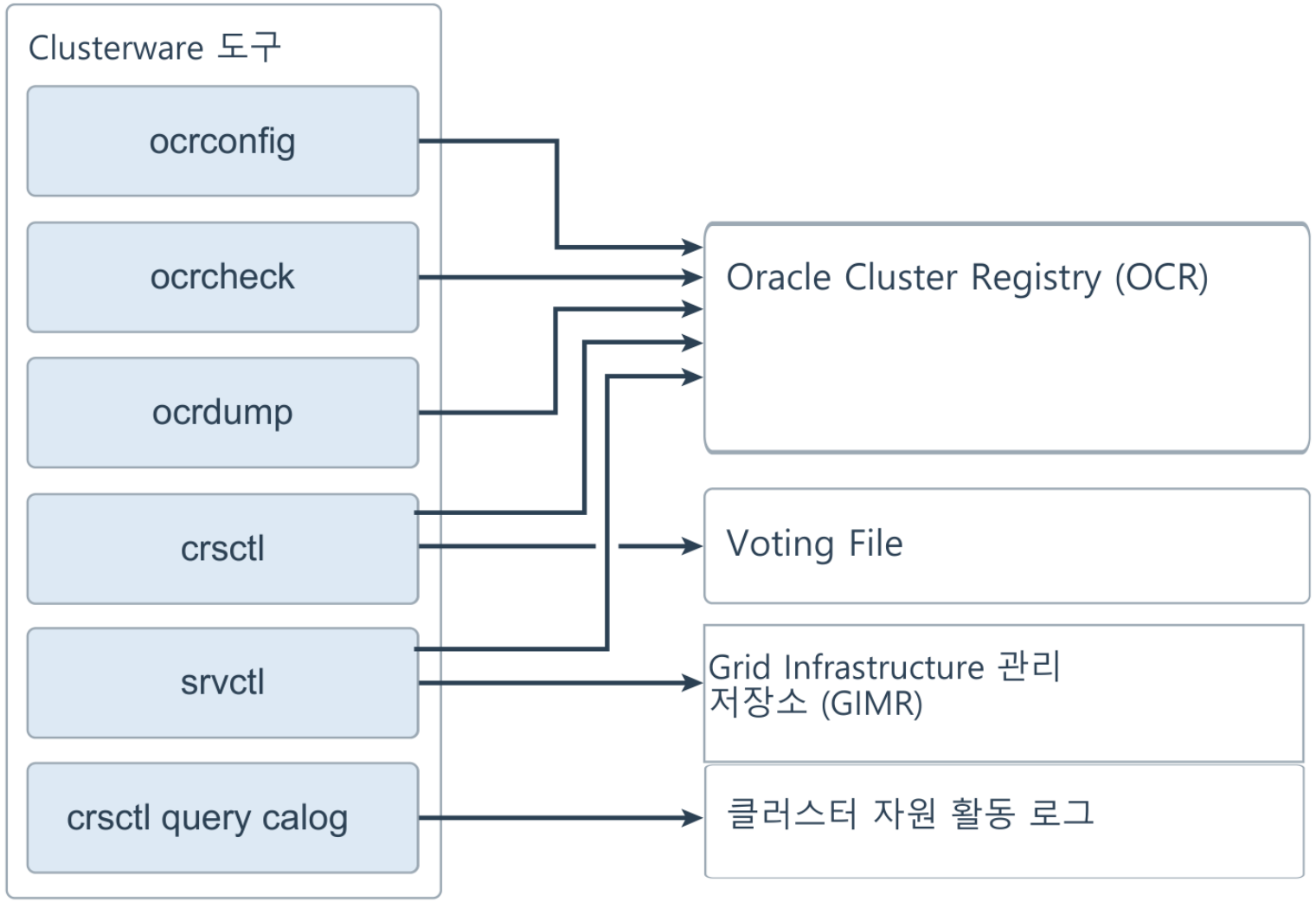
# 2

# srvctl 과 crsctl

- 두 명령의 경계와 CRS-4995
- srvctl 의 명령 체계와 사용 규칙
- crsctl 이 닿는 계층
- 자원 목록 · 속성 · 의존 관계
- 사용자 자원 실측
- 오류 번호 체계

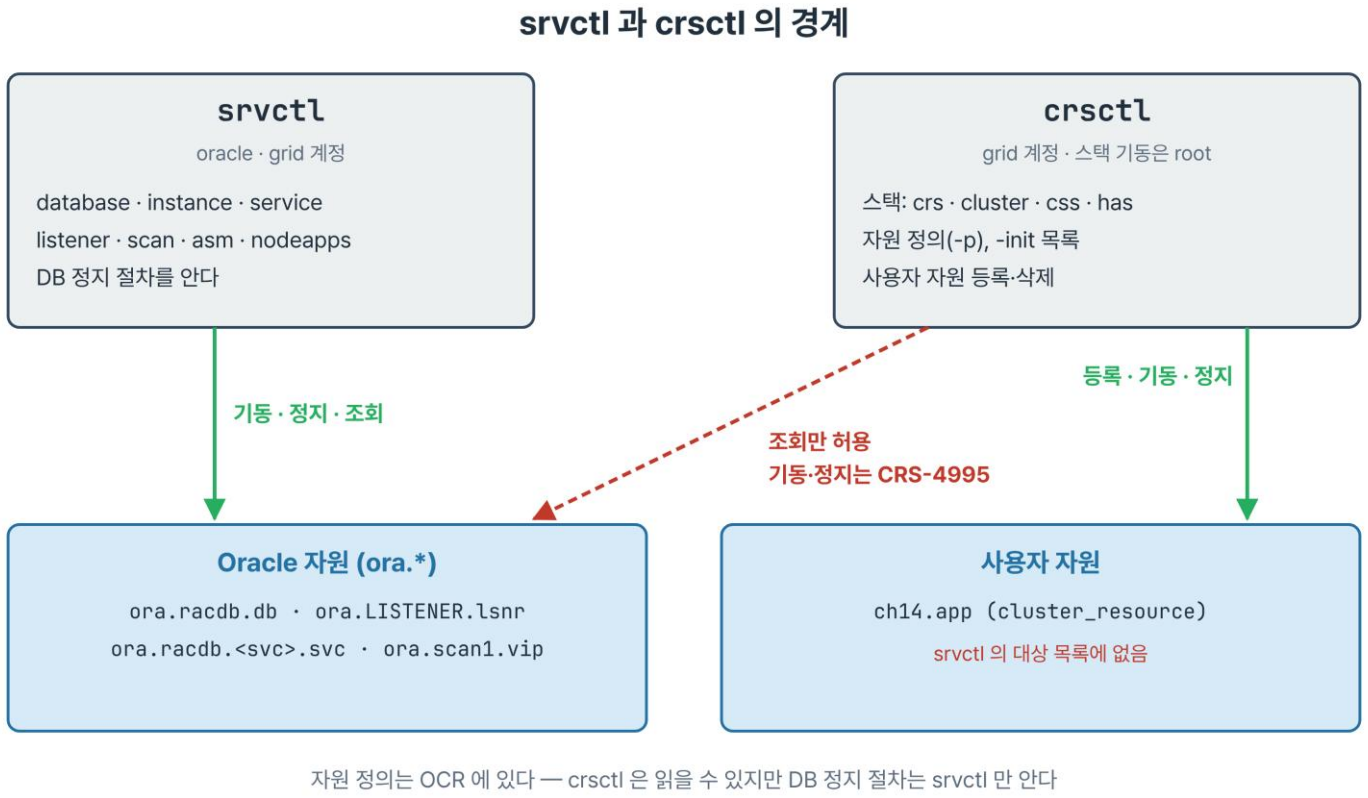
# Clusterware 도구와 GI 파일

crsctl 만 OCR 과 Voting File 양쪽에 닿는다 (원문 26쪽)



# 두 명령의 경계

srvctl 은 Oracle 자원, crsctl 은 스택과 자원 모델




- ora.* 를 crsctl 로 기동 · 정지 → CRS-4995, 종료 코드 1
- 조회(-p)는 허용 — 정의는 OCR 에 있다
- srvctl 은 사용자 자원을 모른다 — 안내도 없다

# srvctl 의 세 토막

srvctl <명령> <대상> <옵션>

- status 는 현실, config 는 정의
- stop 은 지금, disable 은 앞으로
- -eval 은 명령의 영향, predict 는 장애의 영향
- 한 글자 옵션은 12c 부터 폐기 예정 — -help -compatible
- 종료 코드 0 성공 · 1 실패 · 2 경고(바뀔 것 없음)

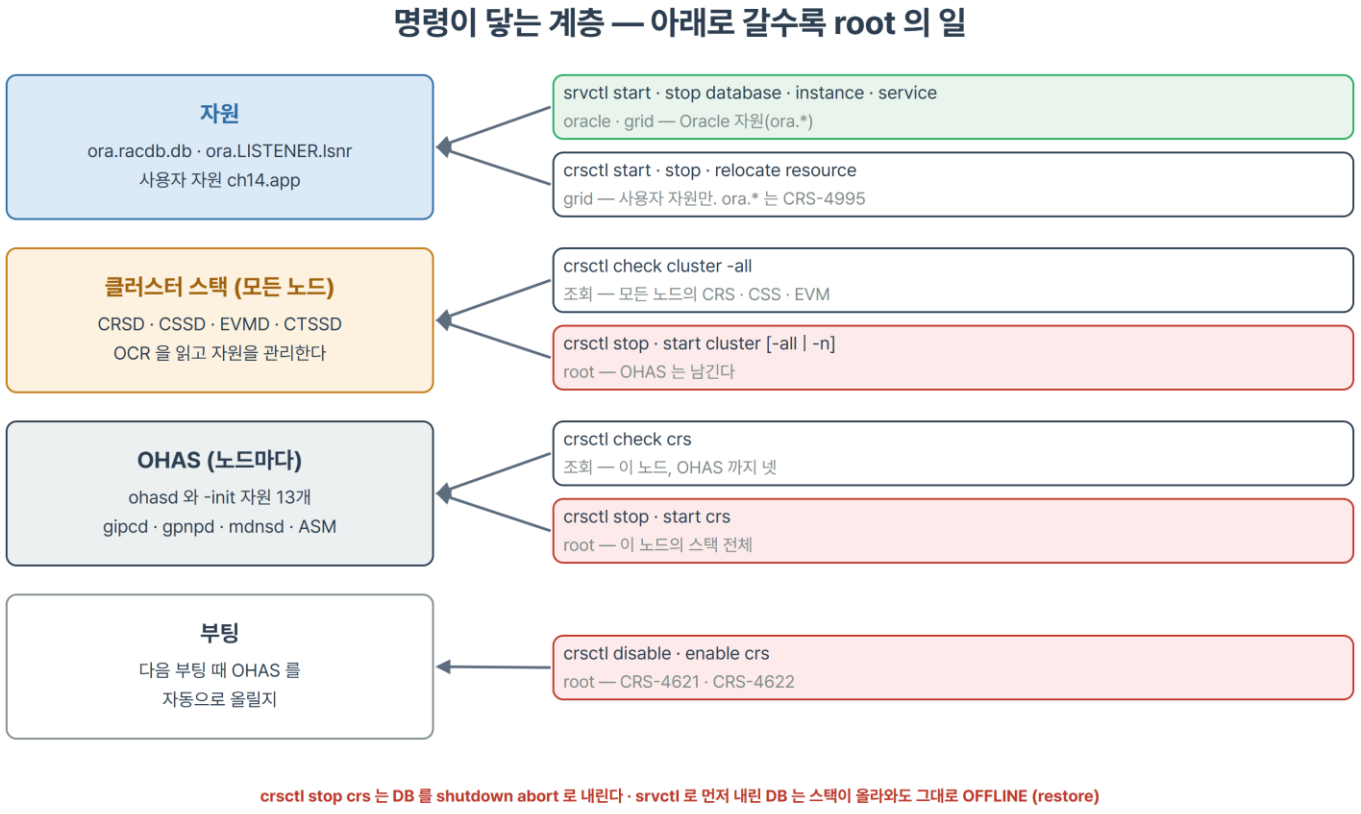
# Management policy

문서의 세 가지

- AUTOMATIC — 재기동 뒤 직전 상태로 (기본)
- MANUAL — 재기동 때 안 올림, 장애 때는 되살림
- NORESTART — 둘 다 안 함
- racdb: AUTOMATIC · Start open · Stop immediate

# crsctl 이 닿는 계층

아래로 갈수록 root 의 일




- check 는 조회, stop · start cluster 는 OHAS 를 남긴다
- stop crs 는 이 노드 전체 — DB 는 shutdown abort
- 여러 노드를 내릴 때는 stop cluster 가 먼저

# crsctl stat res 의 옵션

-t · -p · -v · -dependency · -w

- -t 표, -p 정적 구성, -v 실행 중 값(RESTART_COUNT)
- -dependency [-stop | -pullup] — 의존 관계 목록
- -w "TYPE = ora.database.type" — 연산자 양옆에 공백
- 이름은 정확히 — ora.racdb.database 는 CRS-2613
- crsctl eval start · stop · relocate resource — 미리 보기

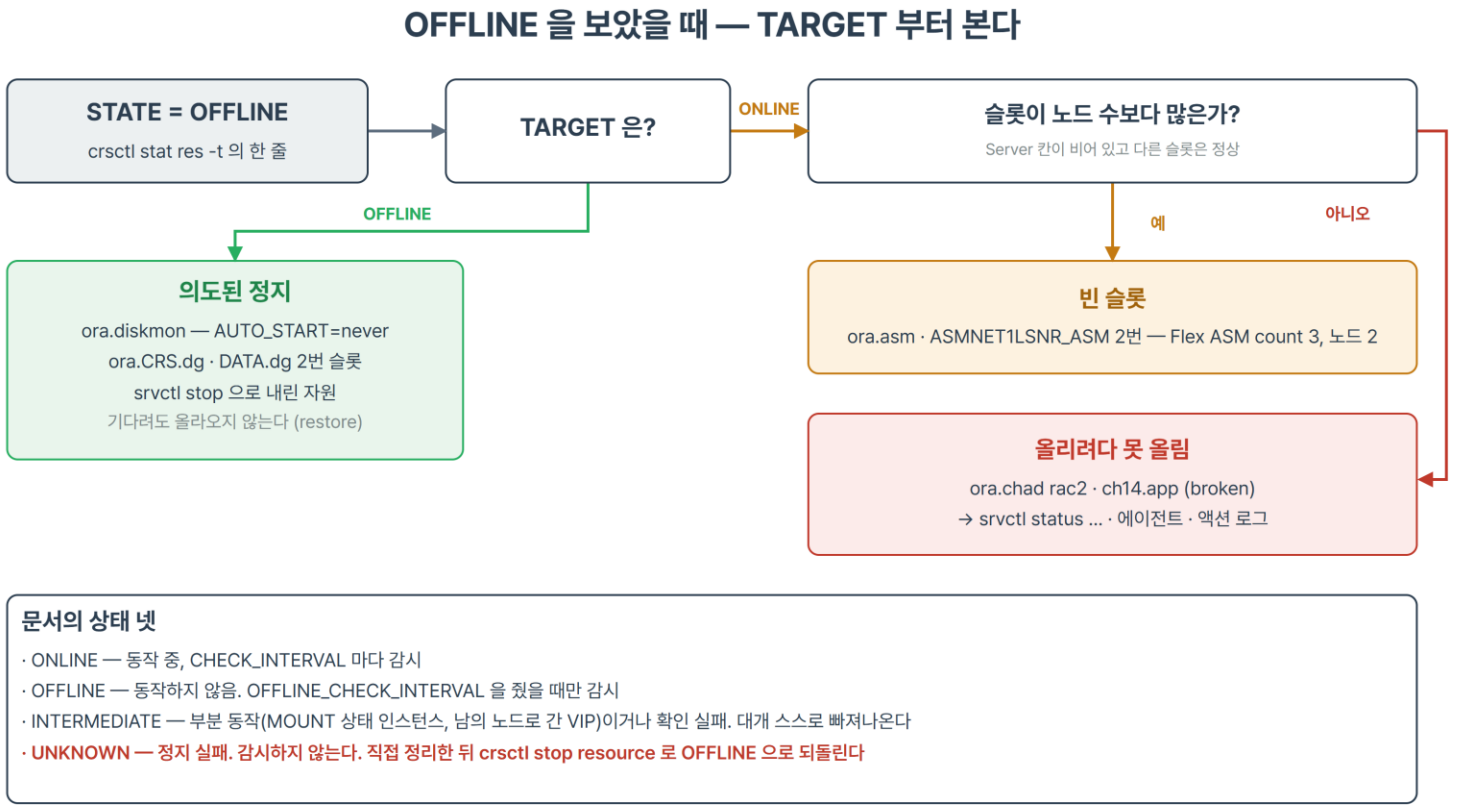
# 자원 목록 읽기

crsctl stat res -t

- Local Resources — 노드마다 하나, 번호 없음
- Cluster Resources — 왼쪽 숫자는 슬롯 번호
- 기준선: 일반 17개, -init 13개
- State details — Open, HOME= 까지 보인다

# OFFLINE 을 보았을 때

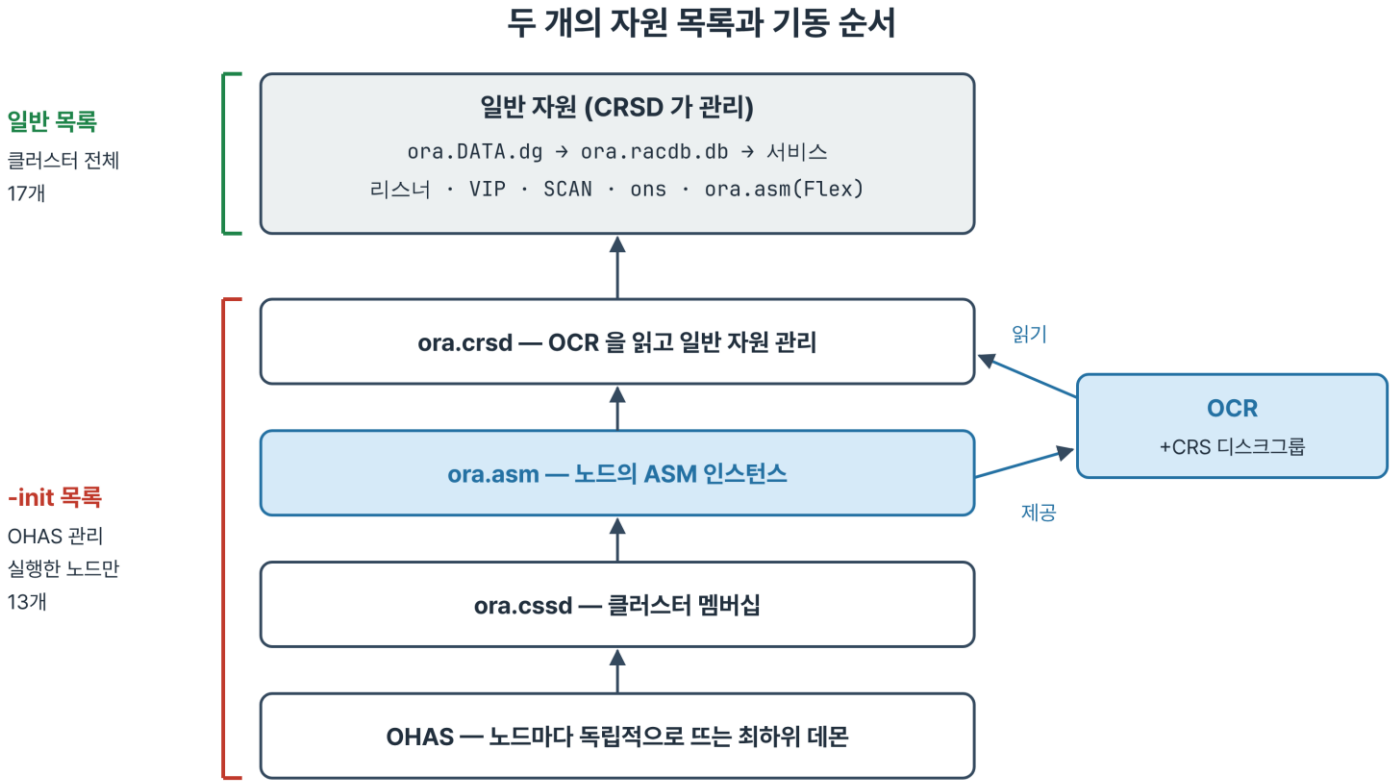
TARGET 부터 본다




- TARGET OFFLINE — 의도된 정지 (diskmon, dg 2번 슬롯)
- 빈 슬롯 — Flex ASM count 3, 노드 2
- 올리려다 못 올림 — ora.chad rac2

# -init 이 보여 주는 하위 계층

OHAS 는 -init 자원을, CRSD 는 일반 자원을




- -init 은 실행한 노드의 것만
- OHAS → CSSD → ASM → CRSD → 일반 자원
- ora.asm 은 두 목록에 모두 — OCR 이 +CRS 에 있어서

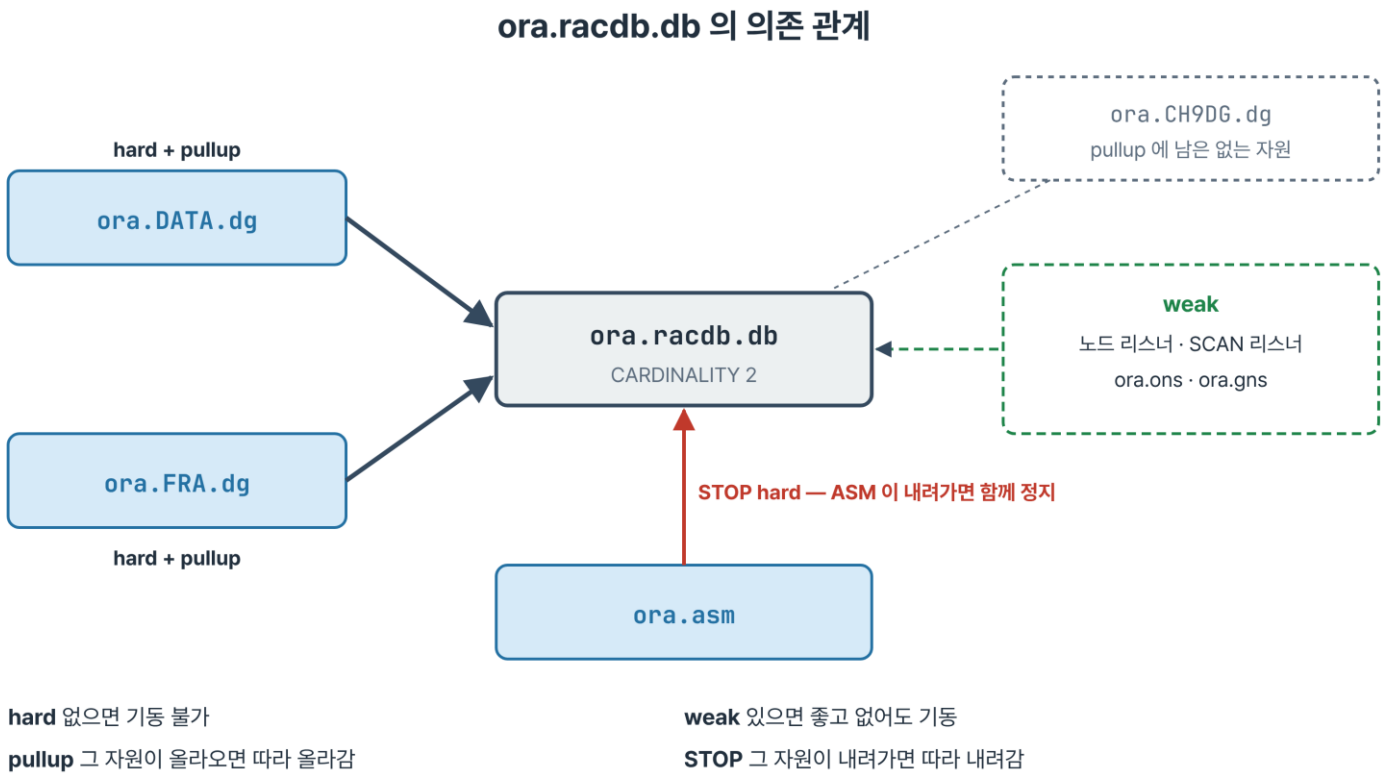
# 자원 속성

crsctl stat res ora.racdb.db -p

- AUTO_START — always · restore · never
- RESTART_ATTEMPTS — VIP 0, DB 2, cssd 3, crsd 10
- CHECK_INTERVAL — VIP · 네트워크 · ASM 1초, DB 900초
- UPTIME_THRESHOLD — 지나면 RESTART_COUNT 를 0 으로

# 의존 관계

hard 는 필수, weak 는 선택, pullup 은 연쇄 기동




- global — 다른 노드에 있어도 된다 (Flex ASM)
- pullup 은 TARGET ONLINE 일 때만 (always 가 없으면)
- 정지 의존 shutdown — 스택을 내릴 때만

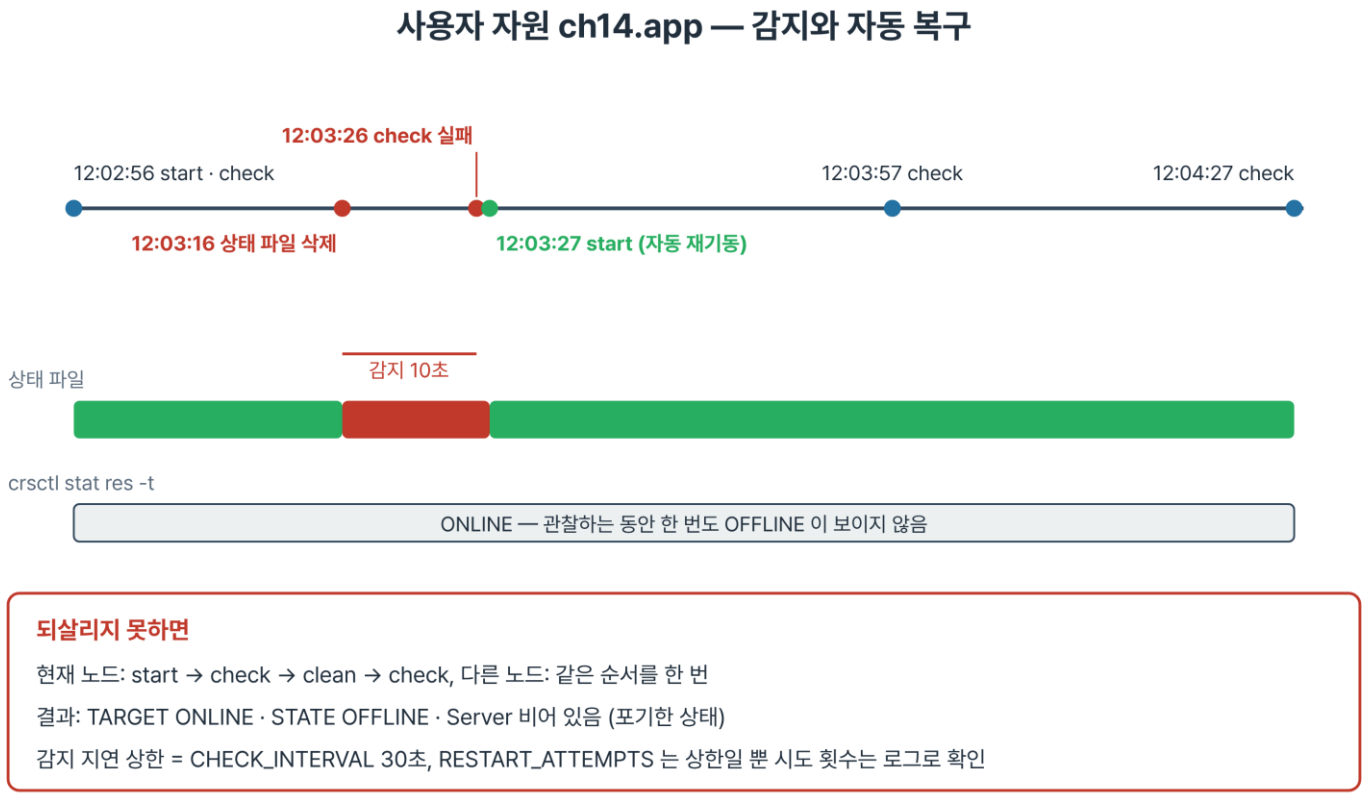
# 사용자 자원 등록

ch14.app — cluster_resource

- 액션 스크립트: start · stop · check · clean
- check 0 = 살아 있음, 그 밖 = 죽음
- PLACEMENT=restricted, HOSTING_MEMBERS=rac1 rac2
- add 는 성공해도 말이 없다 — echo $? 를 본다
- 따옴표가 상쇄되면 CRS-2613 + CRS-4000

# 자동 복구 — 실측

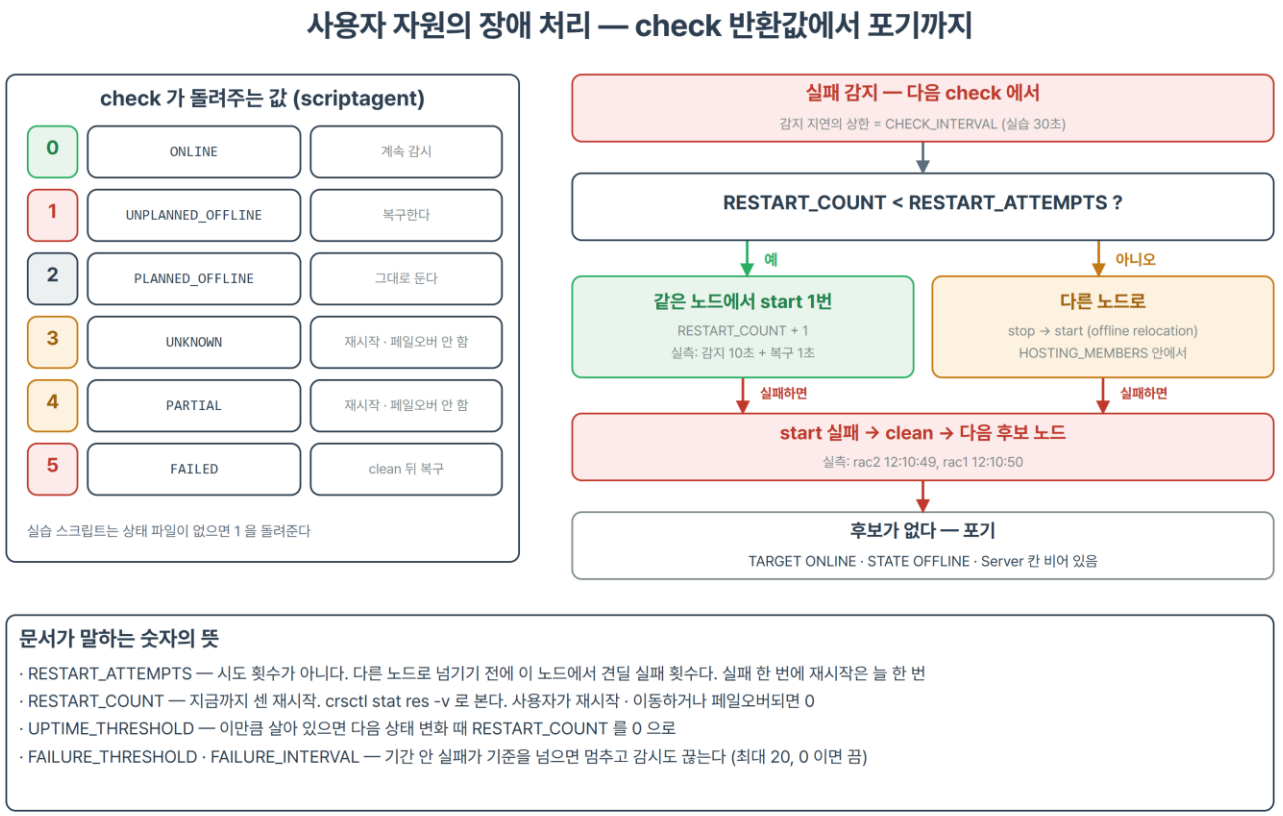
감지 10초 + 복구 1초




- 12:03:16 삭제 → 12:03:26 check 실패 → 12:03:27 start
- crsctl stat res -t 는 한 번도 OFFLINE 을 보이지 않았다
- 감지 지연의 상한 = CHECK_INTERVAL 30초

# 되살리지 못하면

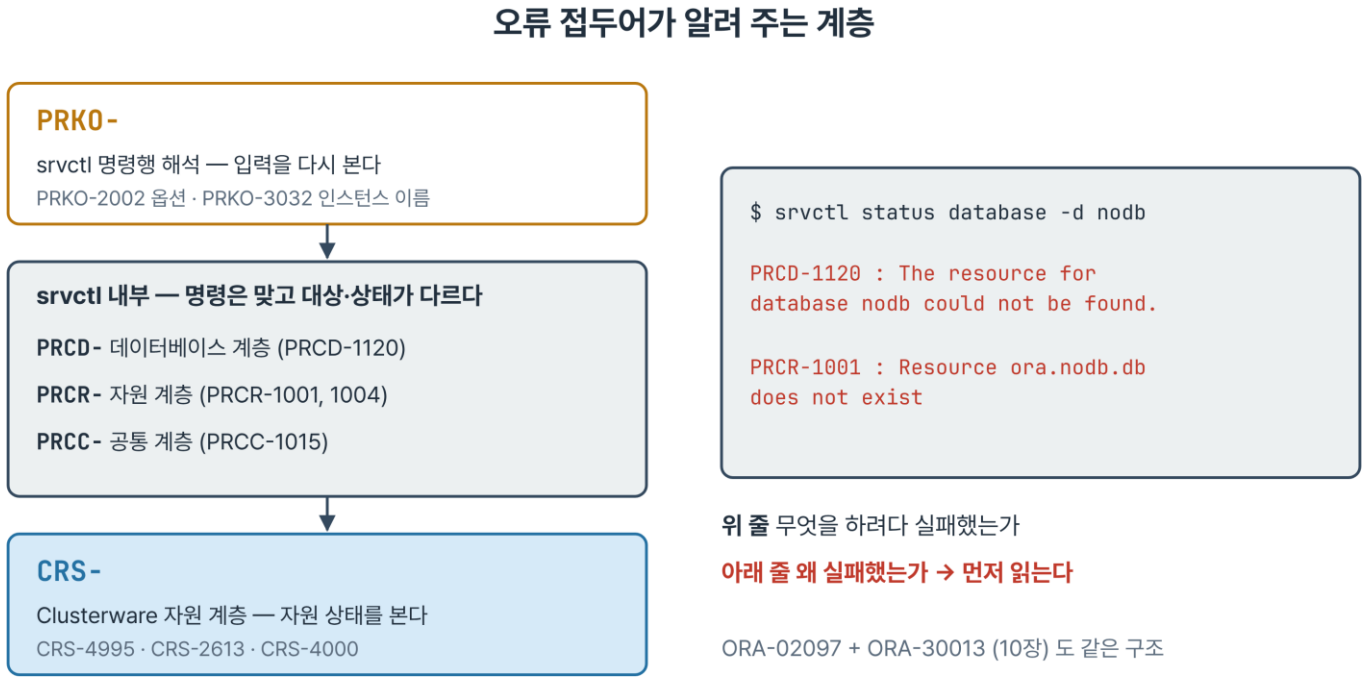
check 반환값에서 포기까지




- rac2 start 실패 → clean → rac1 에서 시도 → 포기 (2초)
- TARGET ONLINE · STATE OFFLINE · Server 칸 비어 있음
- RESTART_ATTEMPTS 는 시도 횟수가 아니라 견딜 실패 횟수

# 오류 번호 체계

겹쳐 나오면 아래 줄이 원인




- PRKO 는 명령행 — 입력을 다시 본다
- PRCD · PRCR · PRCC · CRS 는 대상 · 상태
- PRCC-1015 + PRCR-1004 는 원하는 결과

# 일상 점검

클러스터 → 인스턴스 순서

- 매일 crsctl check cluster -all
- 매일 crsctl stat res -t — 17개, 의도된 OFFLINE 만
- 매일 srvctl status database -d racdb -v
- 주간 노드마다 crsctl stat res -t -init — 13개
- 수시 사용자 자원 -t · -v 와 액션 로그

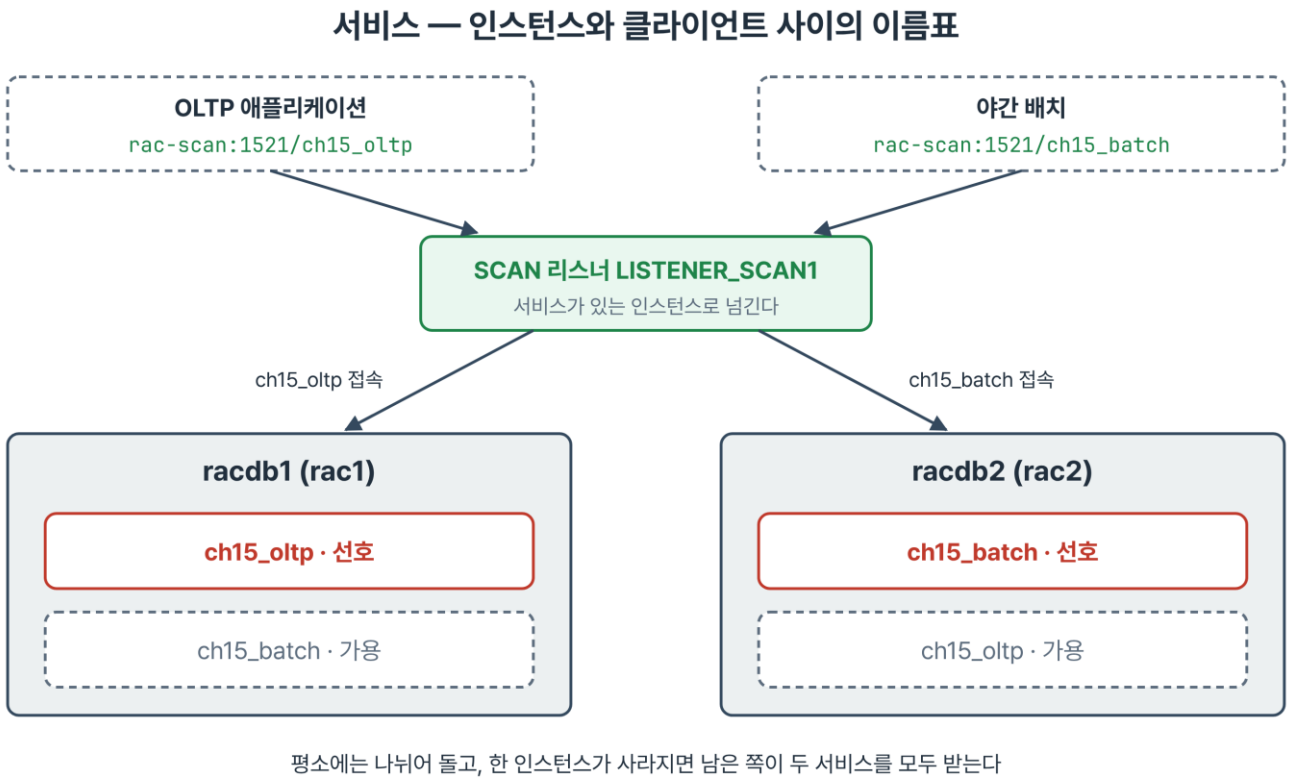
# 3

# 동적 데이터베이스 서비스

- 서비스가 맡는 자리와 리스너 등록
- 서비스 속성과 Clusterware 자원
- relocate 와 드레인
- 장애와 계획 정지, 접속 부하 분산

# 서비스는 이름표다

클라이언트는 인스턴스가 아니라 이름에 붙는다




- 이름 하나가 여러 인스턴스, 인스턴스 하나가 여러 이름
- 정의는 OCR, 실행 상태는 인스턴스와 리스너
- 선호를 엇갈리게, 서로를 가용으로

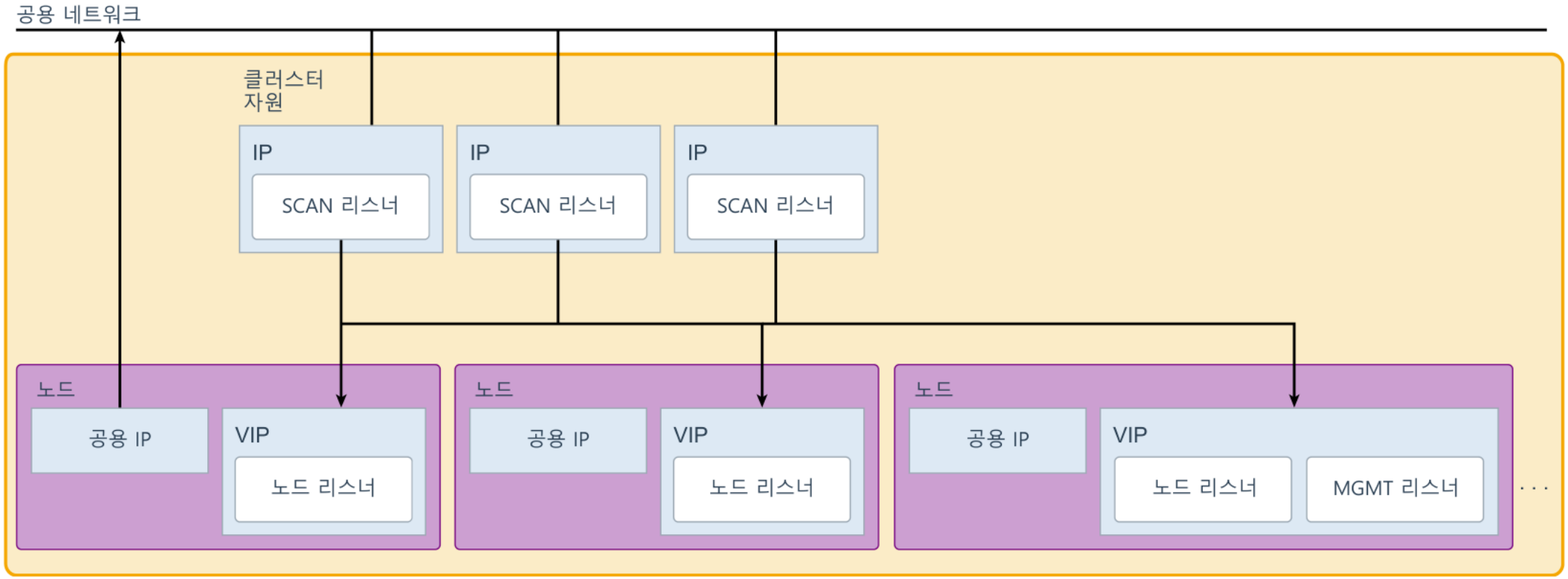
# 이미 있는 서비스

사용자 서비스가 없어도 인스턴스마다 다섯 개

- srvctl status service — 출력 없음, 종료 코드 0
- racdb · racdbXDB · racpdb1(CON_ID 3)
- SYS$BACKGROUND · SYS$USERS — 내부 서비스
- 기본 · 내부 서비스는 옮기거나 멈출 수 없다

# RAC 의 리스너

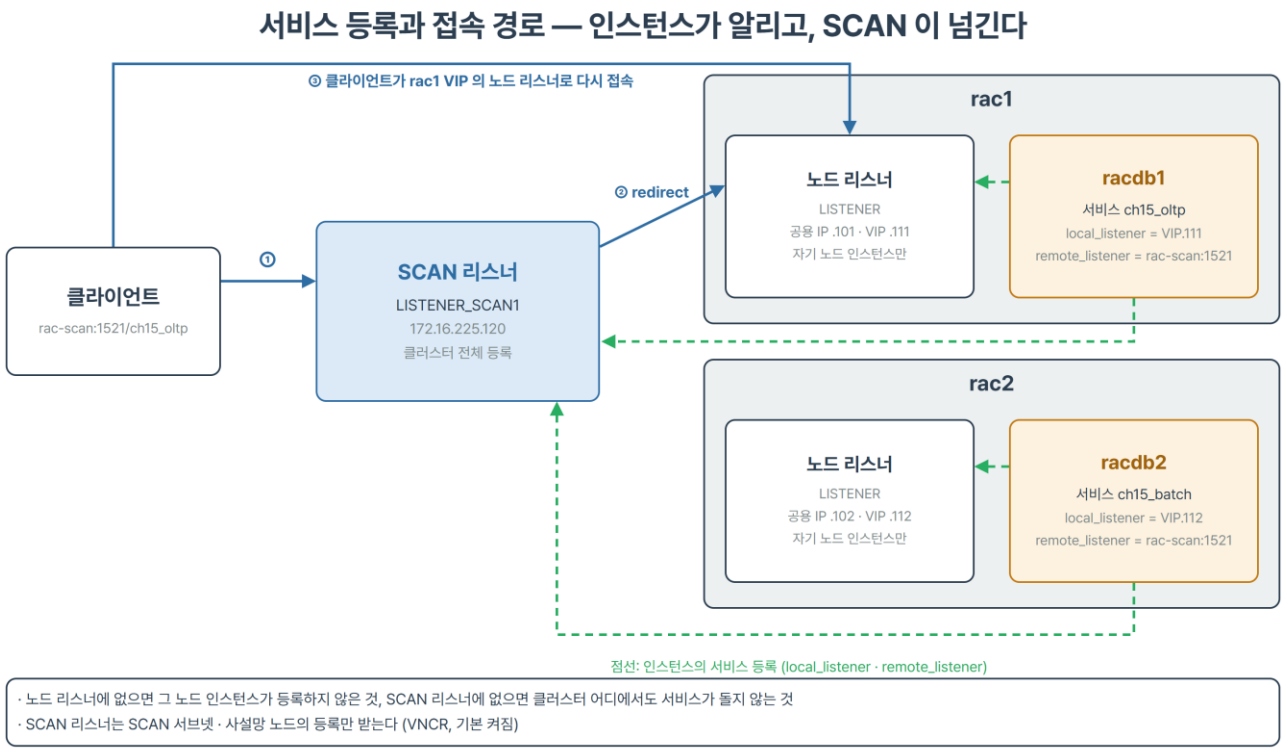
SCAN 리스너, 노드 리스너, MGMT 리스너 (원문 20쪽)




- local_listener = 자기 노드 VIP (.111 · .112)
- remote_listener = rac-scan:1521 (두 인스턴스 같음)
- 노드 리스너는 자기 노드 인스턴스만 안다

# 등록과 접속 경로

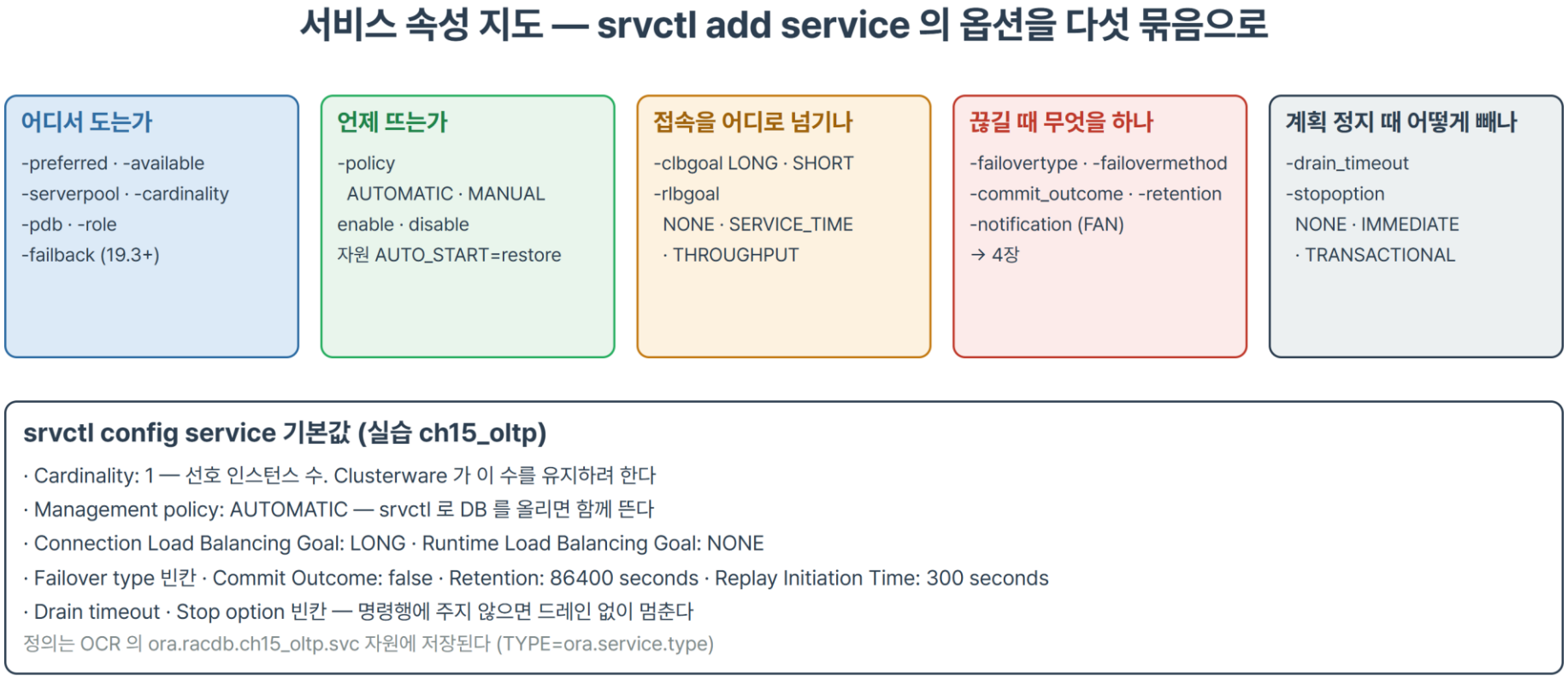
인스턴스가 알리고, SCAN 이 넘긴다




- ① 클라이언트 → SCAN 리스너
- ② 서비스를 가진 인스턴스의 노드 리스너로 redirect
- ③ 클라이언트가 그 노드 VIP 로 다시 접속
- VNCR — SCAN 은 정해진 서브넷의 등록만 받는다

# 서비스 속성 지도

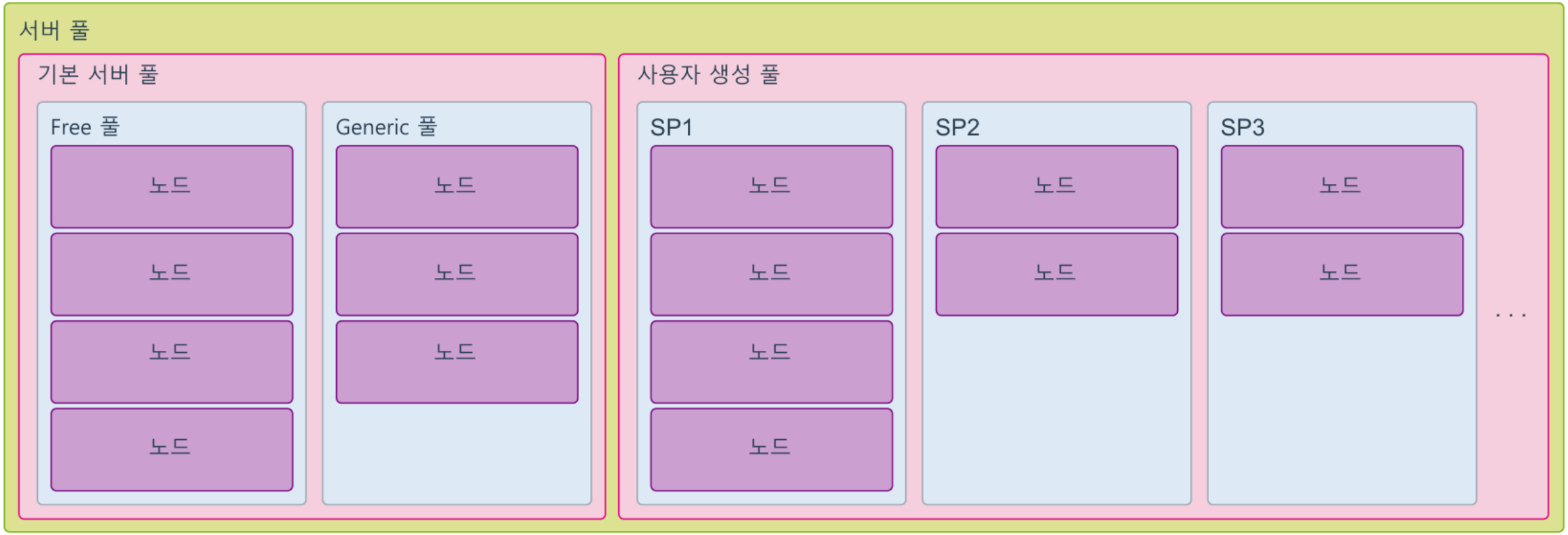
srvctl add service 의 옵션을 다섯 묶음으로




- 어디서 · 언제 · 어디로 넘기나 · 끊길 때 · 뺄 때
- -failback YES — 19.3 부터 선호 인스턴스로 자동 복귀
- Retention 86400 · Replay 300 — 문서 기본값

# 서버 풀

정책 관리형 DB 는 인스턴스 대신 풀을 지정한다 (원문 38쪽)




- 관리자 관리형: -preferred · -available (실습 racdb)
- 정책 관리형: -serverpool, UNIFORM · SINGLETON
- relocate 도 -oldinst/-newinst 대 -currentnode/-targetnode

# 서비스는 Clusterware 자원이다

ora.racdb.ch15_oltp.svc

- 자원 17 → 19개, 만들기만 하면 OFFLINE / OFFLINE
- TYPE=ora.service.type · AUTO_START=restore
- 프로필에 인스턴스 · DB 와의 의존이 들어 있다
- PRKO-2101(명령행) · PRCD-1120 + PRCR-1001(자원)

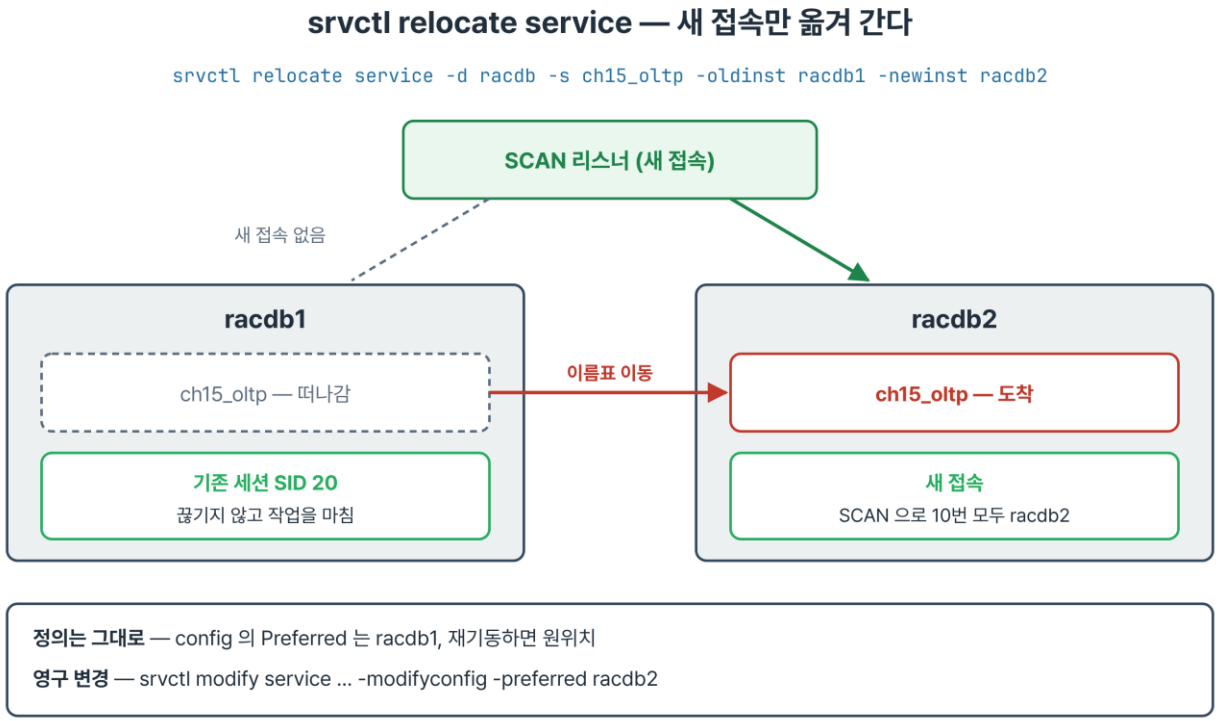
# 배치는 네 곳에서 확인한다

Clusterware · 인스턴스 · 리스너 · 클라이언트

- srvctl status database -v — 인스턴스와 서비스를 한 번에
- GV$ACTIVE_SERVICES — 인스턴스가 연 서비스
- lsnrctl status LISTENER_SCAN1 — has 1 instance(s)
- SCAN 으로 10번 접속 → 10번 racdb1

# relocate

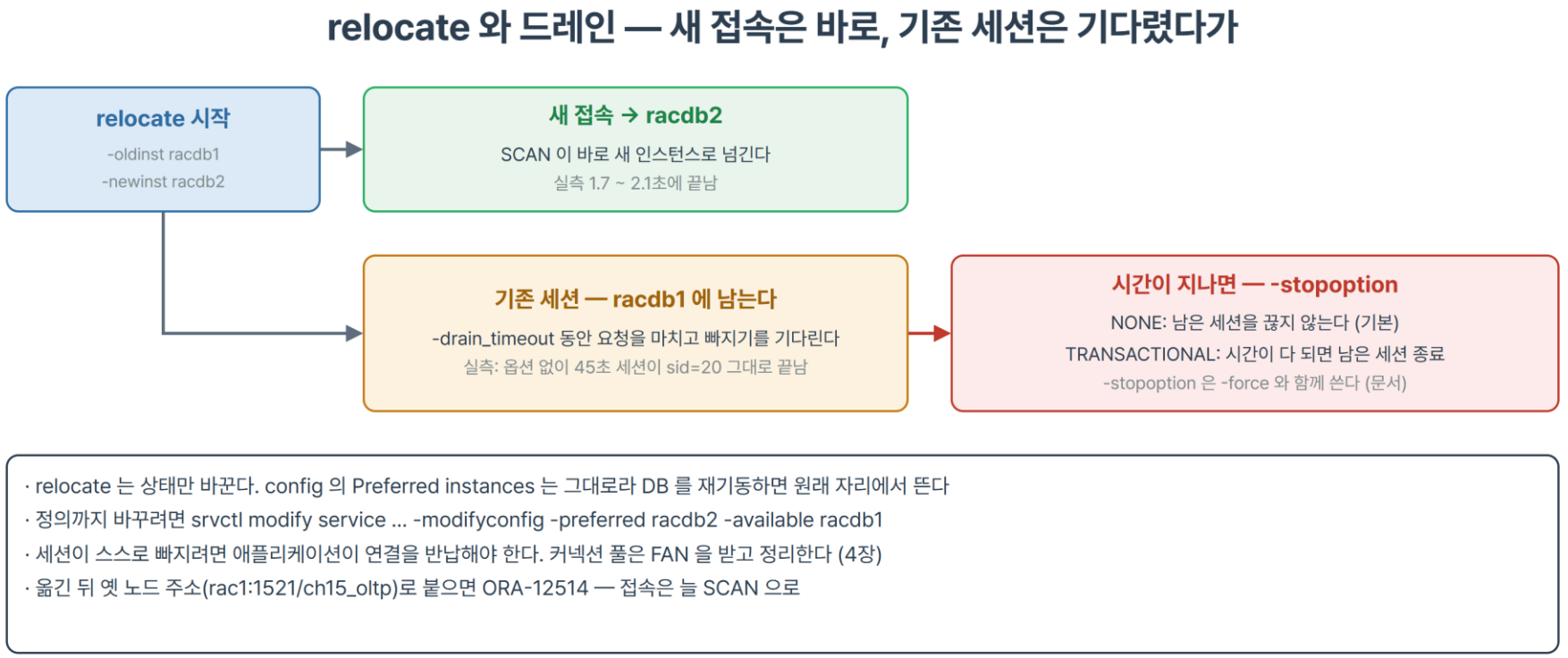
새 접속의 방향만 바꾼다




- 이동 2.1 · 2.0 · 1.7초
- 직전 세션: BEFORE · AFTER 모두 racdb1 sid=20
- config 의 Preferred 는 그대로 — 재기동하면 원위치
- 옛 노드 주소로 붙으면 ORA-12514

# 드레인

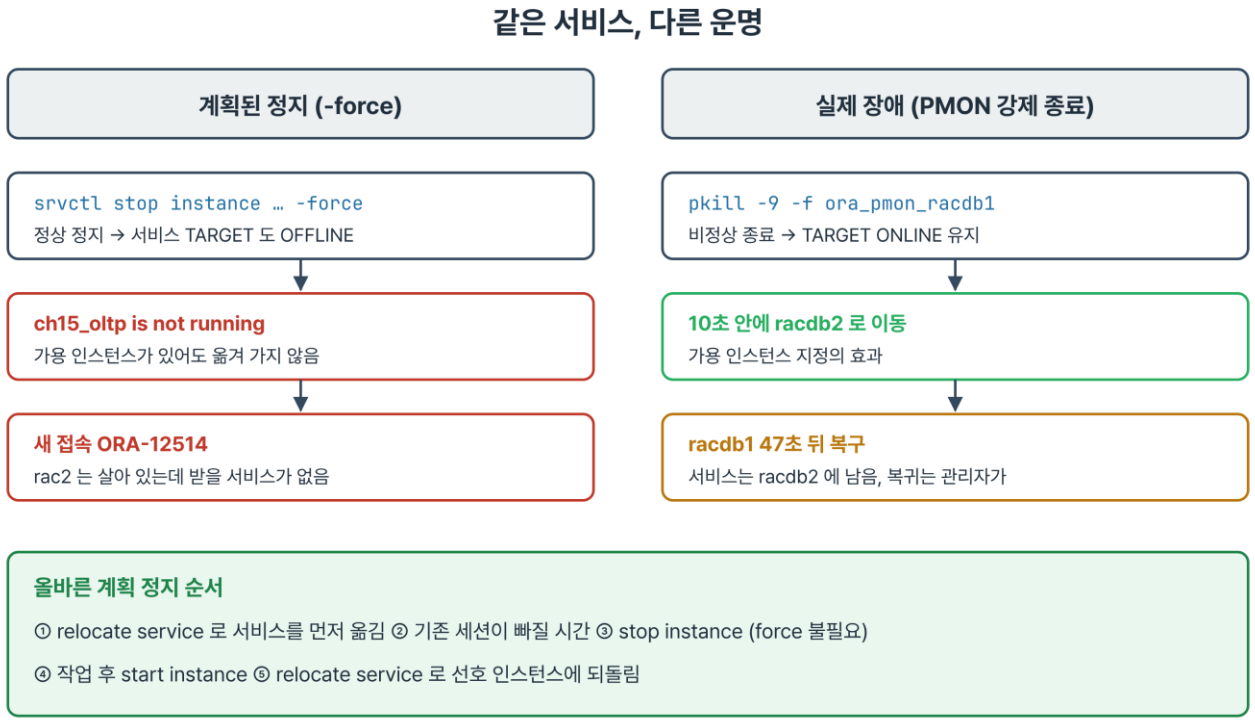
새 접속은 바로, 기존 세션은 기다렸다가




- -drain_timeout — 다 빠지면 남은 시간을 기다리지 않는다
- -stopoption NONE(기본) · IMMEDIATE · TRANSACTIONAL
- 문서: -stopoption 은 -force 와 함께
- 세션이 빠지려면 풀이 연결을 반납해야 한다 (4장)

# 계획 정지와 실제 장애

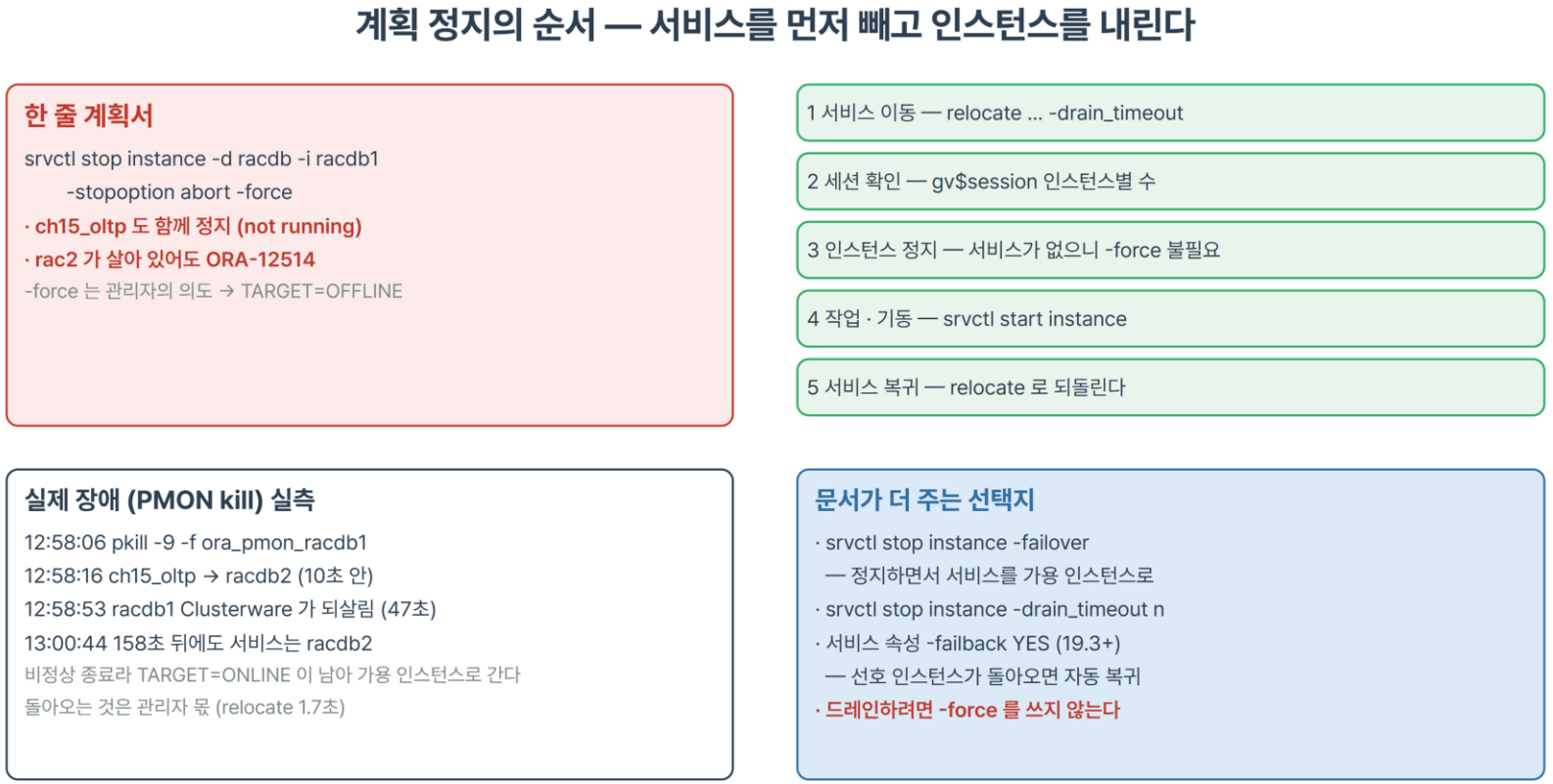
TARGET 이 서비스의 운명을 가른다




- 그냥 stop instance → CRS-2974 (네 줄 중 마지막)
- -force → ch15_oltp not running, ORA-12514
- PMON kill → 10초 안에 racdb2 로
- racdb1 은 47초 뒤 복귀, 서비스는 158초 뒤에도 racdb2

# 계획 정지의 순서

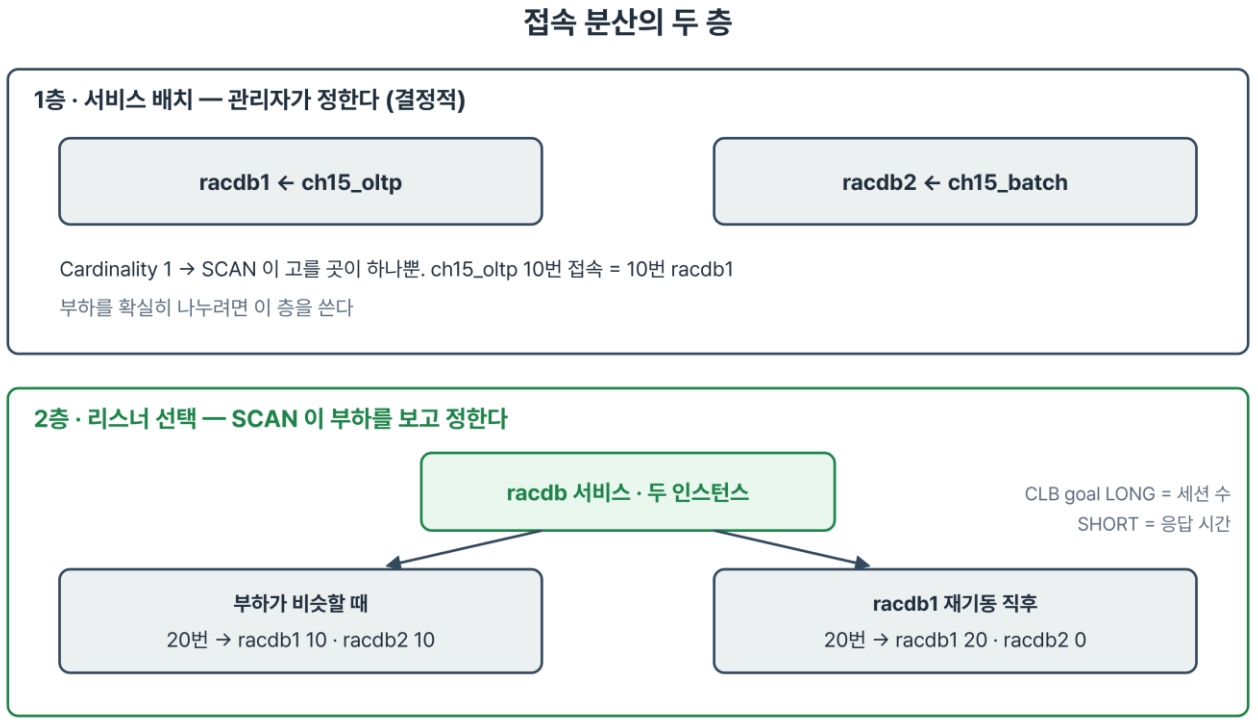
서비스를 먼저 빼고 인스턴스를 내린다




- relocate(드레인) → 세션 확인 → stop instance → 작업 → 복귀
- stop instance -failover · -drain_timeout (문서)
- -failback YES 로 자동 복귀 (19.3+)

# 접속 분산의 두 층

서비스 배치는 관리자가, 인스턴스 선택은 리스너가




- racdb 20번: 10 : 10, racdb1 재기동 직후 20 : 0
- CLB goal LONG = 세션 수, SHORT = 응답 시간 · 처리량
- 문서 권장은 LONG
- SCAN 은 세 IP 를 다 시도한 뒤 실패를 돌려준다

# 서비스로 보는 성능

세션과 SQL 대신 서비스와 SQL

- V$SERVICE_STATS · V$SERVICEMETRIC
- DBMS_MONITOR.SERV_MOD_ACT_STAT_ENABLE
- 병렬 서버는 그 서비스의 인스턴스에만
- SYS$USERS 14 — 서비스 없이 붙은 세션부터 찾는다

# 운영 지침

ORA-12514 는「모른다」이지「없다」가 아니다

- 정지 · 인스턴스 장애 · 이름 오타 · 노드 주소 접속
- 매일: status 와 config 의 Preferred 비교
- 서비스는 애플리케이션마다, 선호와 가용을 함께
- 정리: stop → remove, 자원 17개로 복귀

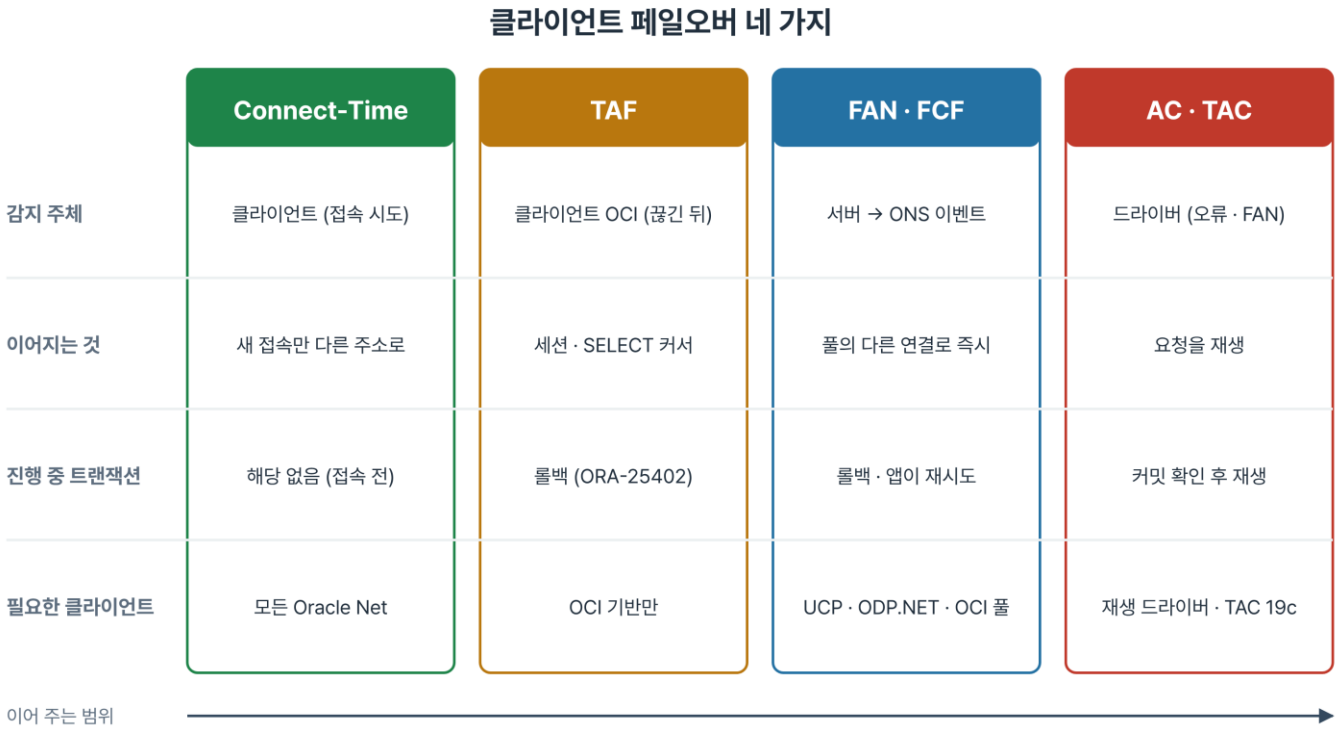
# 4

# FAN · TAF · Application Continuity

- 클라이언트 쪽 고가용성의 네 층
- FAN 과 ONS, FCF
- 부하 분산 권고
- TAF · Transaction Guard · AC · TAC
- 계획 정비 드레이닝

# 클라이언트 페일오버 네 가지

무엇이 감지하고 어디까지 이어 주는가




- Connect-Time Failover — 새 접속만
- TAF — 끊긴 뒤 재접속, SELECT 는 커서까지
- FAN · FCF — 서버가 먼저 알린다
- AC · TAC — 진행 중 요청을 재생

# TCP 타임아웃과 FAN

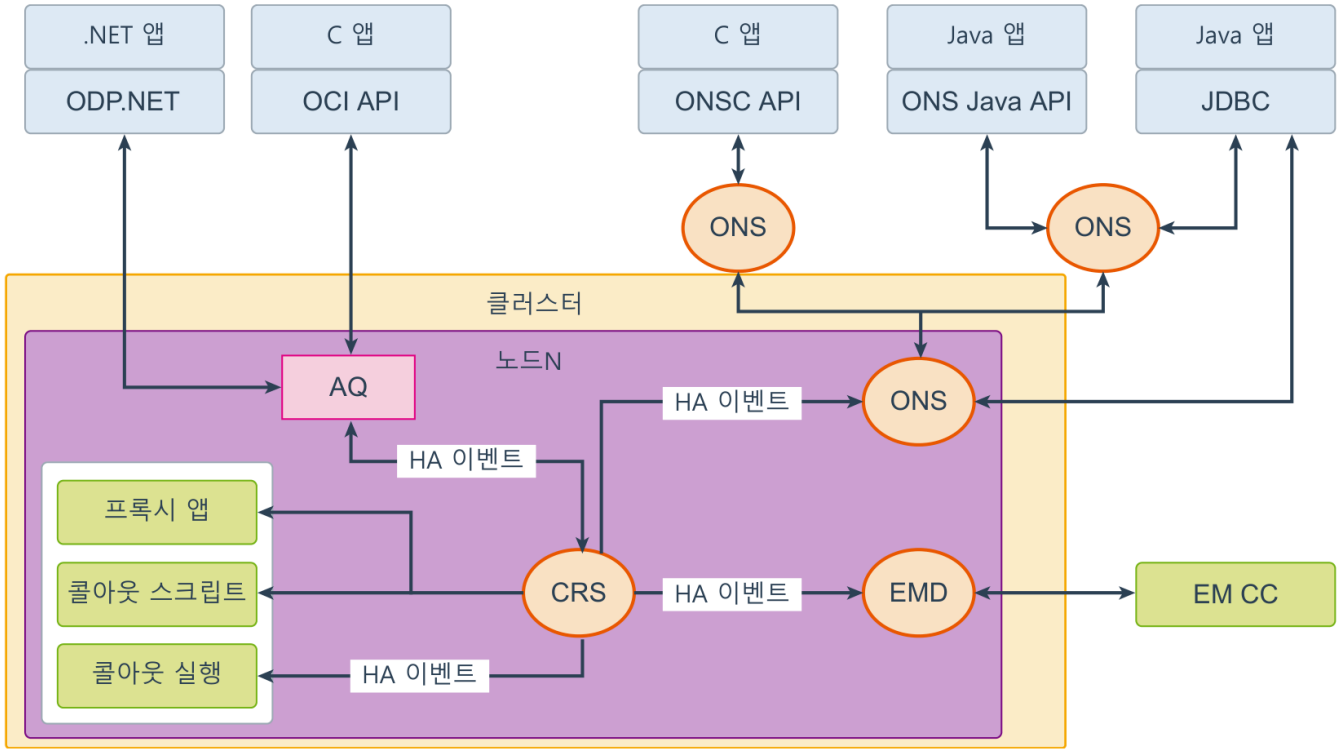
장애를 누가 먼저 아는가




- FAN 없음: keepalive 를 기다린다 — 수 분
- FAN 구독 풀: 죽은 연결 즉시 정리
- 이벤트는 살아 있는 노드가 발행한다

# ONS 와 HA 이벤트

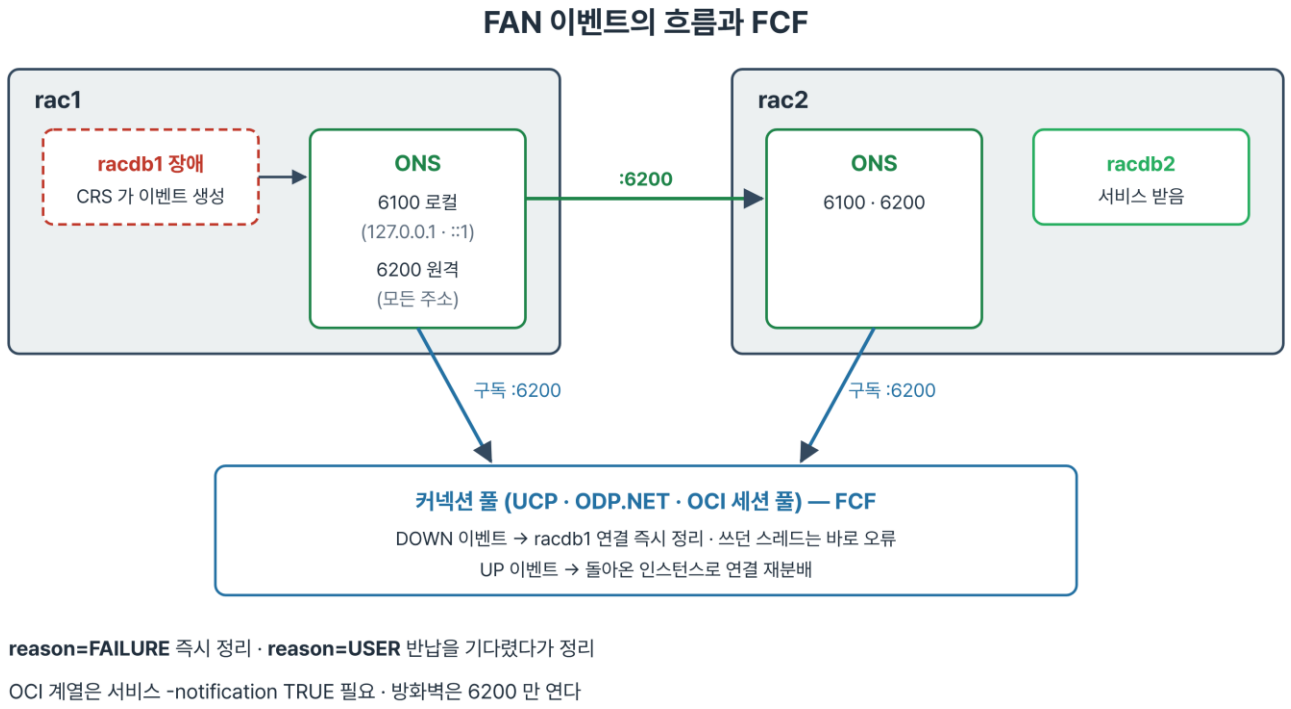
CRS 가 만든 이벤트의 세 갈래 (원문 35쪽)




- ONS — Java · C 클라이언트
- AQ — ODP.NET · OCI (하위 호환)
- 콜아웃 · EM
- 리스너도 죽은 인스턴스의 서비스를 즉시 등록 해제

# ONS 의 두 포트

로컬 6100, 원격 6200 — 실측




- 6100: 127.0.0.1 · ::1 에만 — 같은 노드
- 6200: 모든 주소 — 다른 노드 ONS · 원격 클라이언트
- 방화벽은 6200 만
- onsctl debug 의 Servers: 와 DEFER

# FAN 이벤트의 형식

이름=값 한 줄

- 유형: SERVICEMEMBER · SERVICE · INSTANCE · DATABASE · NODE · SERVICEMETRICS
- status: UP · DOWN · NODEDOWN · NOT_RESTARTING · UNKNOWN
- reason=FAILURE → 즉시 정리, reason=USER → 반납 뒤 정리
- UP 이벤트로 돌아온 인스턴스에 다시 연결
- 콜아웃: Grid_home/racg/usrco, 모든 노드에

# Connect-Time Failover

접속 기술자만으로 동작한다

- SCAN 이름 → IP 세 개, 실패하면 다음 IP
- TRANSPORT_CONNECT_TIMEOUT=3 · CONNECT_TIMEOUT · RETRY_COUNT
- 실습 SCAN 은 IP 하나 — SCAN VIP 가 옮겨 가서 계속 접속된다
- 죽은 노드의 VIP 는 접속을 즉시 거절한다

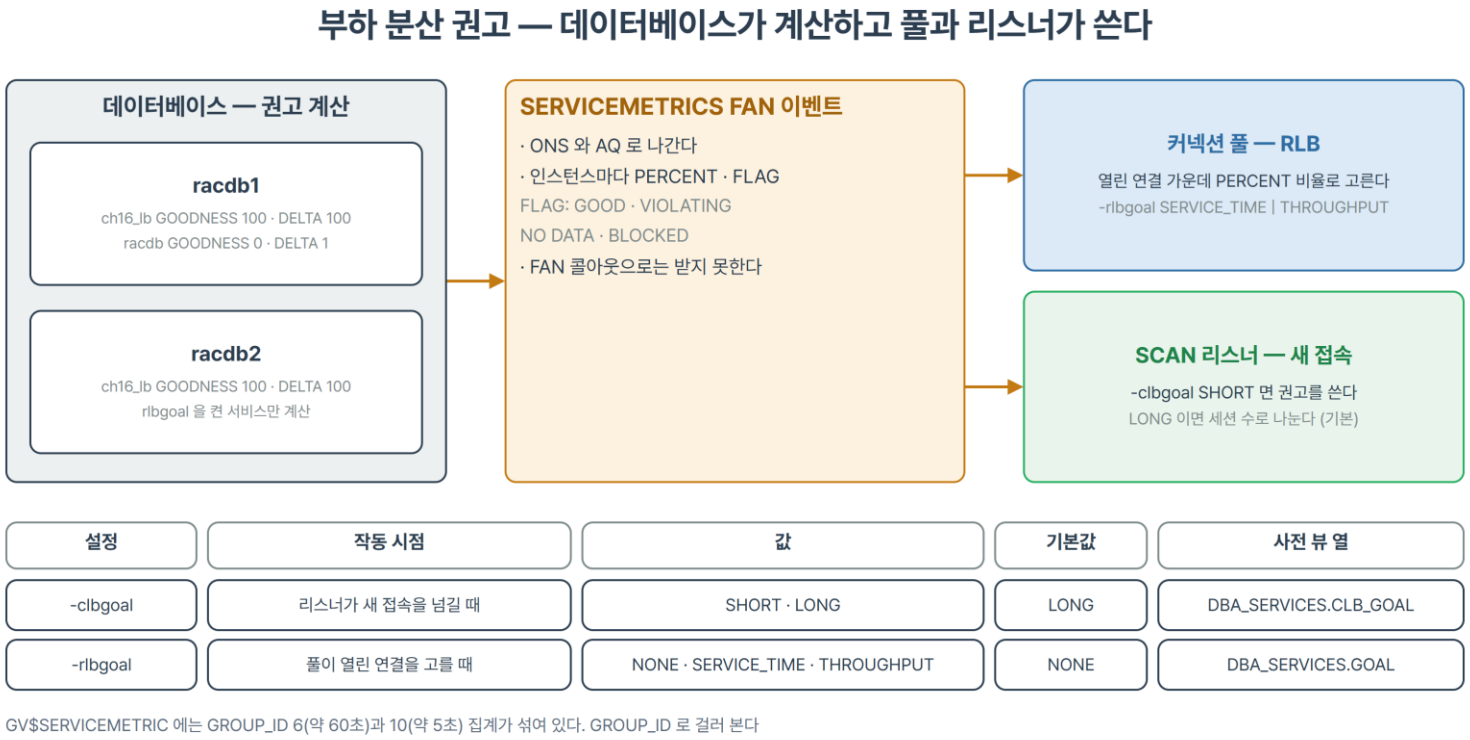
# FCF

커넥션 풀이 FAN 을 쓰는 방식

- UCP: setFastConnectionFailoverEnabled(true)
- ODP.NET: HA Events=true
- OCI: 서비스 -notification TRUE (기본 FALSE)
- 진행 중 작업은 살리지 않는다 — 오류 후 재시도
- JDBC 에 FCF 를 켰으면 TAF 를 함께 쓰지 않는다

# 부하 분산 권고

데이터베이스가 계산하고 풀과 리스너가 쓴다



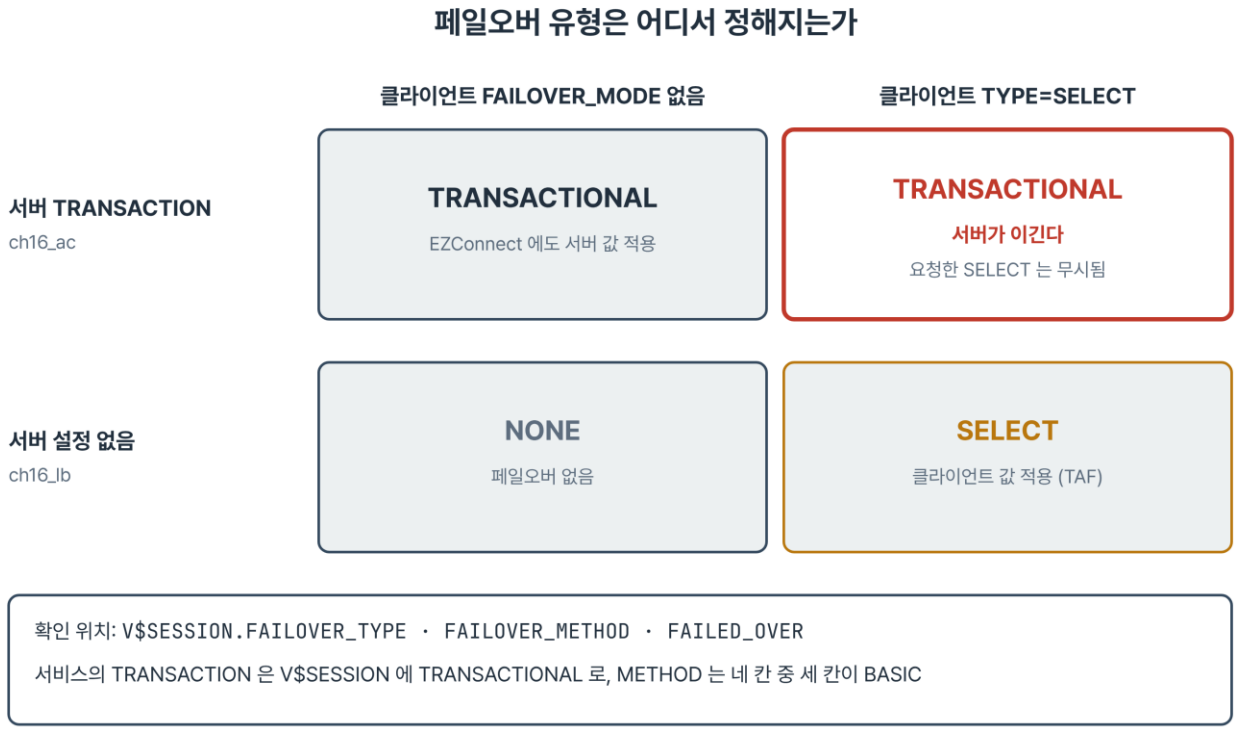

| 설정 | 작동 시점 | 값 | 기본값 | 사전 뷰 열 |
|---|---|---|---|---|
| -clbgoal | 리스너가 새 접속을 넘길 때 | SHORT · LONG | LONG | DBA_SERVICES.CLB_GOAL |
| -rlbgoal | 풀이 열린 연결을 고를 때 | NONE · SERVICE_TIME · THROUGHPUT | NONE | DBA_SERVICES.GOAL |

GV$SERVICEMETRIC 에는 GROUP_ID 6(약 60초)과 10(약 5초) 집계가 섞여 있다. GROUP_ID 로 걸러 본다

- -rlbgoal SERVICE_TIME | THROUGHPUT 이면 권고 계산
- SERVICEMETRICS 이벤트: PERCENT · FLAG
- RLB 는 -clbgoal SHORT 와 짝
- GOODNESS 0 은「계산 안 함」

# TAF — 서버가 이긴다

네 칸 실험 (실습 06)




- 서비스만 → 서비스 값, 클라이언트만 → 클라이언트 값
- 둘 다 → 서비스 값 (SELECT 요청 → TRANSACTIONAL)
- 확인은 설정 파일이 아니라 V$SESSION
- -failovermethod 는 하위 호환용

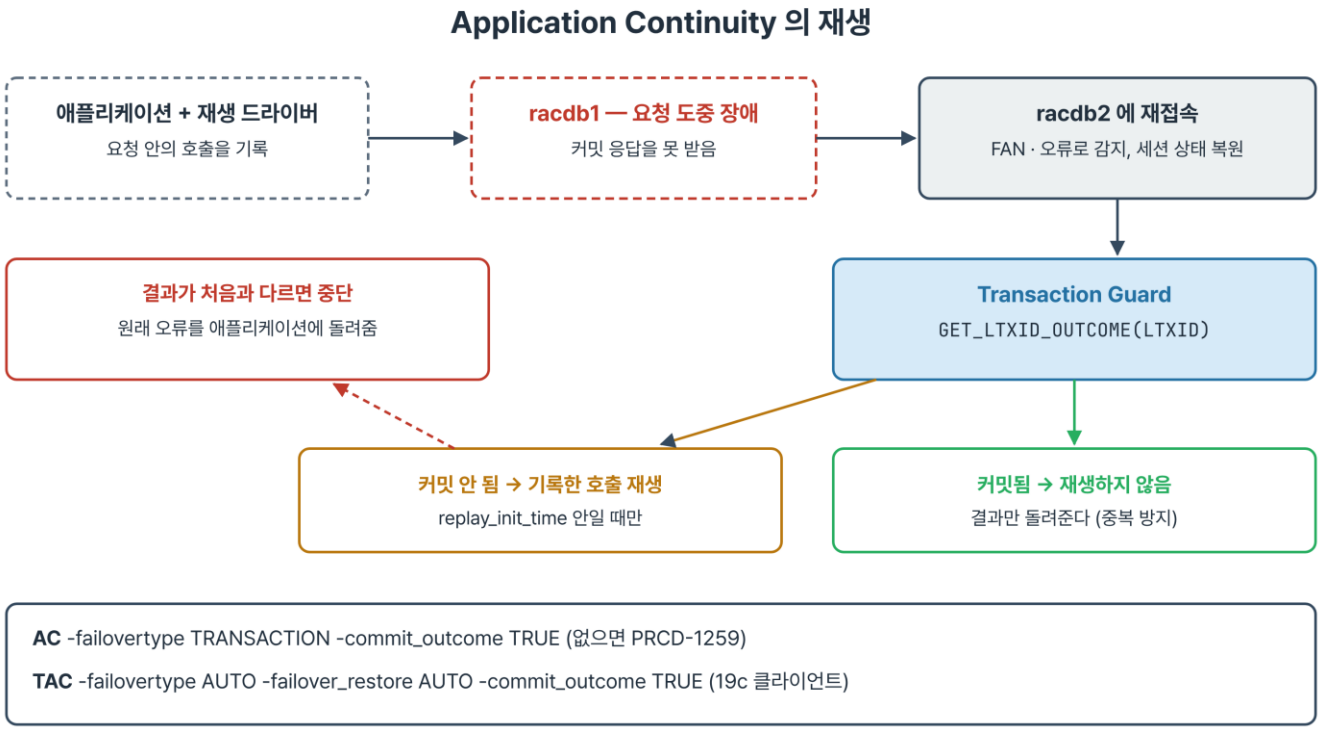
# Transaction Guard

커밋했는지 물을 수 있게

- LTXID — 커밋되면 영속, 롤백되면 재사용
- DBMS_APP_CONT.GET_LTXID_OUTCOME
- -commit_outcome TRUE · -retention 86400
- 이력: LTXID_TRANS (SYSAUX)
- 기본 서비스에는 켜지 않는다

# Application Continuity

커밋 여부를 확인한 뒤 재생한다




- 새 세션 → Transaction Guard 확인 → 초기 상태 복원
- 호출 재실행, 단계마다 결과 검증
- 커밋은 마지막 호출에서만
- 다르면 재생 중단, 원래 오류

# AC 서비스 — 실측

ch16_ac

- -failovertype TRANSACTION -commit_outcome TRUE
- -failover_restore LEVEL1 · -session_state DYNAMIC
- -replay_init_time 600 · -retention 86400
- commit_outcome 없이 TRANSACTION → PRCD-1259
- 없는 값 → PRKO-3125

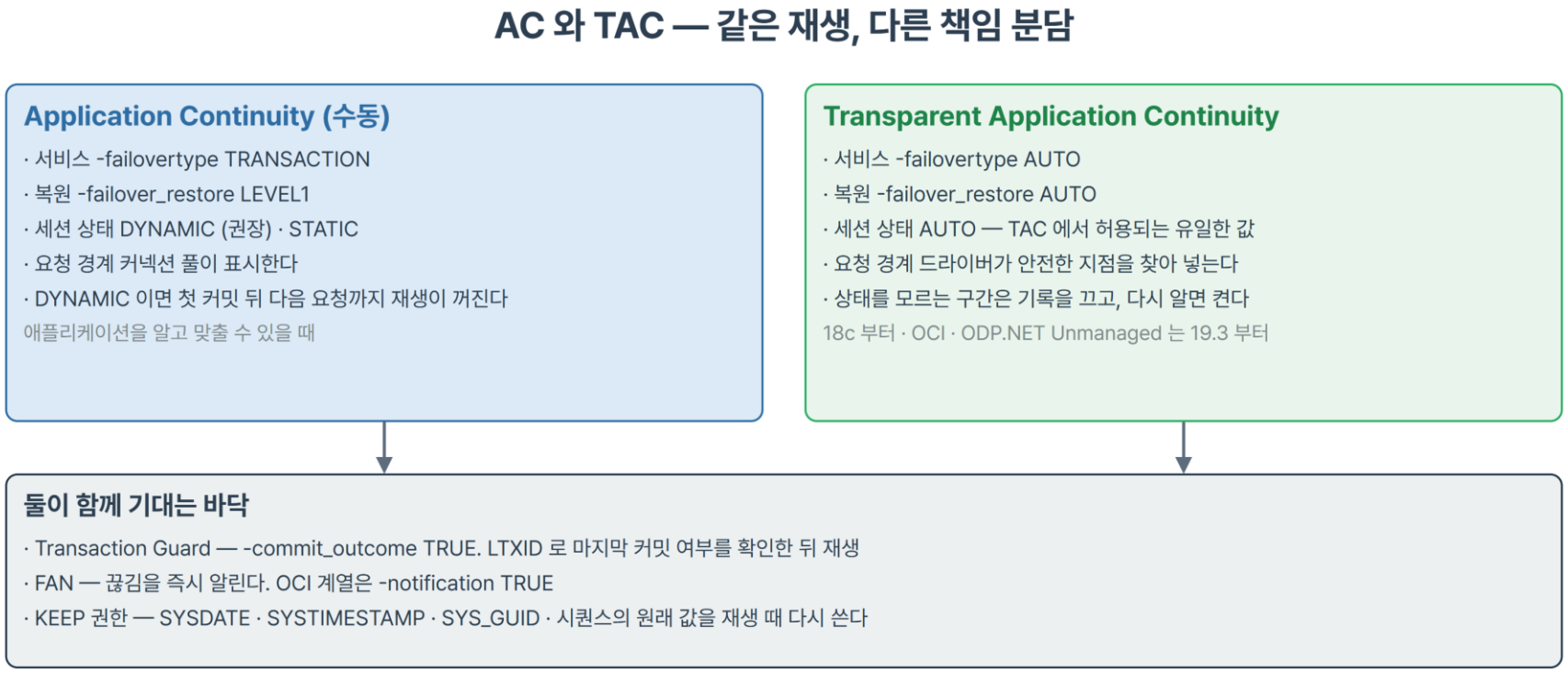
# 재생되지 않는 것

AC 를 켜기 전에 애플리케이션을 본다

- ALTER SYSTEM · ALTER DATABASE
- UTL_HTTP · UTL_FILE · 자율 트랜잭션 — 두 번 일어난다
- SYSDATE · SYS_GUID · 시퀀스 — KEEP 권한
- SERIALIZABLE · Java DRCP · 병렬 쿼리 문장 실패
- 제외 드라이버: JDBC OCI · ODP.NET Managed · ODBC · Pro*C

# AC 와 TAC

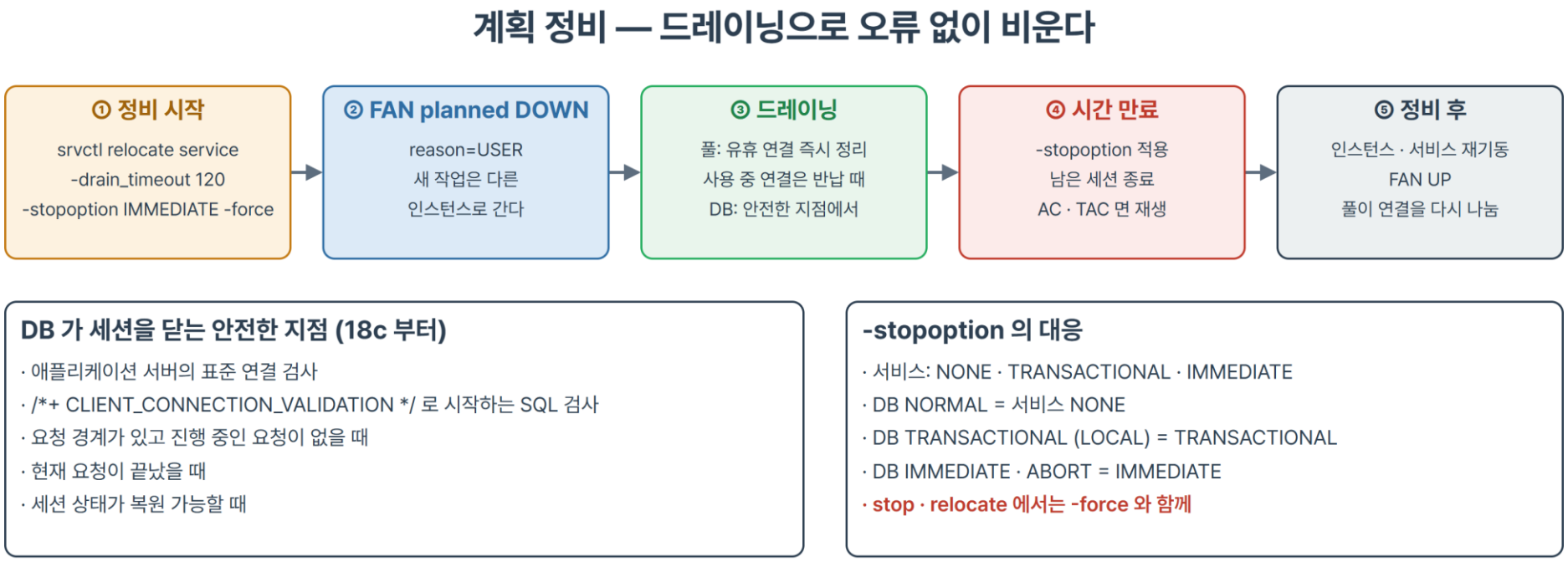
같은 재생, 다른 책임 분담




- TAC: -failovertype AUTO -failover_restore AUTO
- session_state 는 AUTO 만
- 드라이버가 암묵적 요청 경계를 찾는다
- 바닥은 같다: Transaction Guard · FAN · KEEP

# 계획 정비 — 드레이닝

오류 없이 비운다




- relocate · stop service -drain_timeout -stopoption -force
- FAN planned DOWN (reason=USER)
- 풀: 유휴 즉시, 사용 중은 반납 때 · DB: 안전한 지점에서
- 시간 만료 → stopoption, AC · TAC 가 나머지를 덮는다

# 운영 지침

점검 주기

- 매일: onsctl ping (양 노드)
- 주간: onsctl debug 의 Servers · DEFER, srvctl config 와 DBA_SERVICES
- 변경 후: V$SESSION 의 FAILOVER_TYPE
- 분기: 시험 환경 장애 재현, 보호 통계
- 정리: 서비스 stop → remove, tnsnames.ora 341바이트로 복원

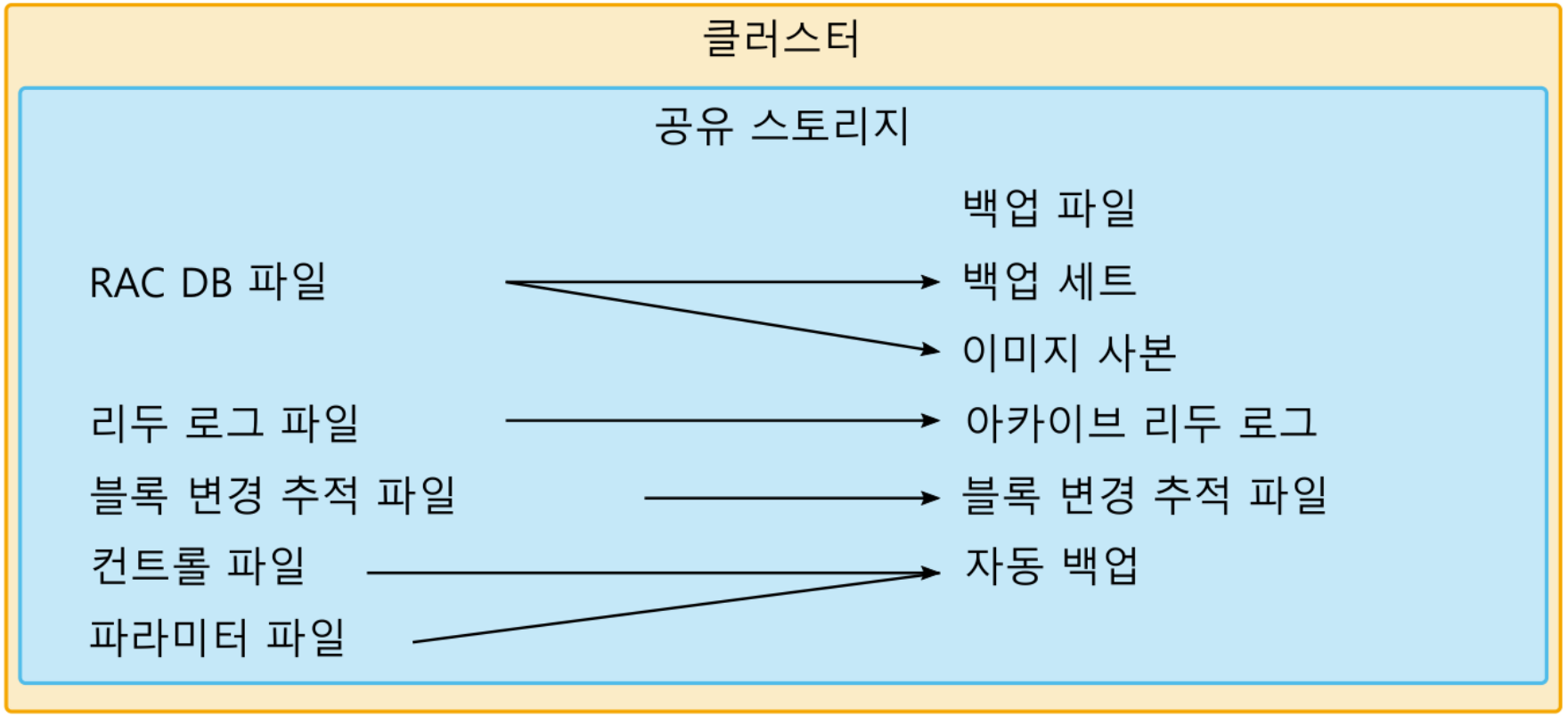
# 5

# RAC 백업과 복구

- 한 벌인 것과 스레드마다 따로인 것
- ARCHIVELOG 와 아카이브 위치
- RMAN 구성 · 채널 · 검증
- 인스턴스 복구와 미디어 복구
- 컨트롤 파일 · SPFILE · FRA

# RAC 의 백업 파일

모두 공유 스토리지에 둔다 (원문 18쪽)




- 데이터 파일 · 컨트롤 파일 · SPFILE 은 한 벌
- 리두와 아카이브는 스레드마다 따로
- 어느 노드든 모든 백업과 모든 스레드를 읽어야 한다

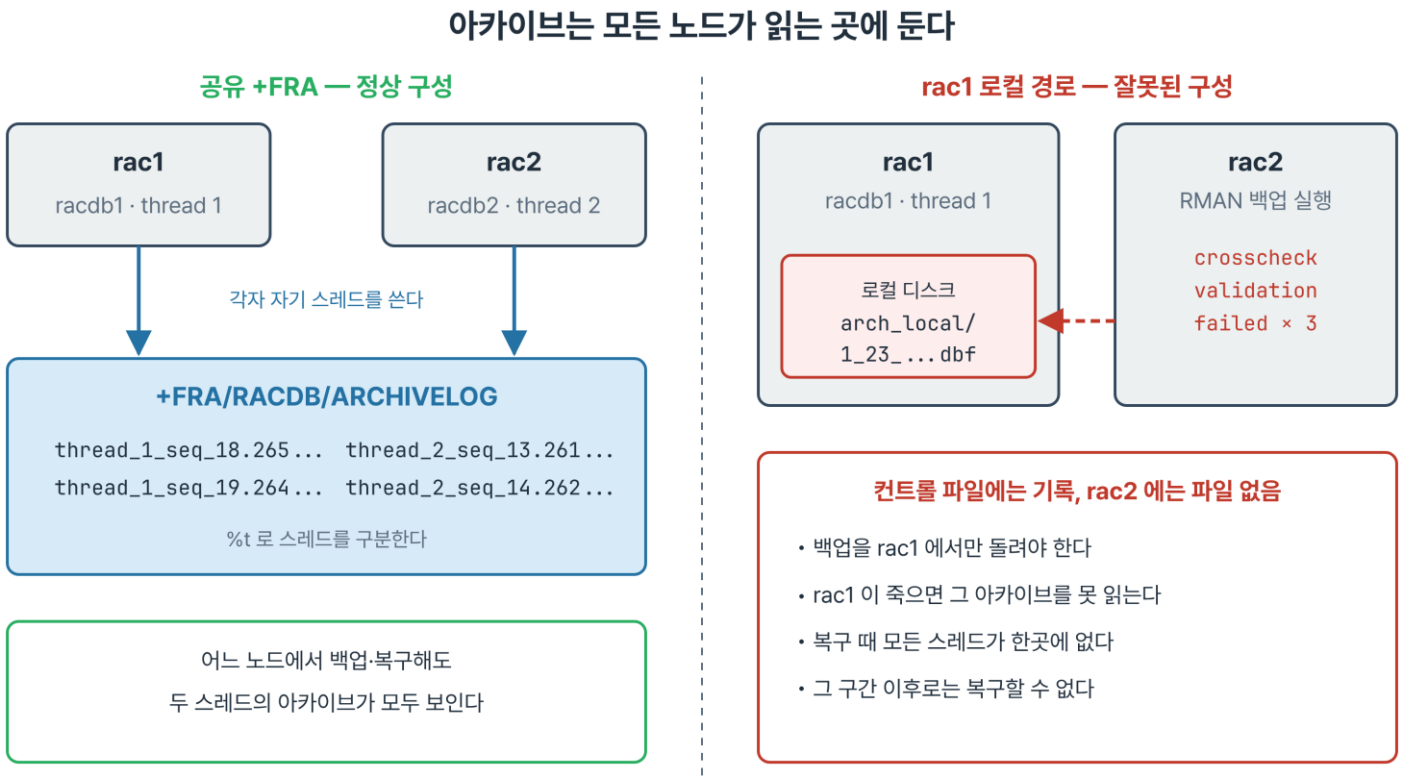
# ARCHIVELOG 전환

데이터베이스 단위 속성, 한 인스턴스만 MOUNT

- srvctl stop database → startup mount → alter database archivelog
- alter database open → srvctl start instance -i racdb2
- 정지 65.7초 · MOUNT 31.8초 · 명령 1.2초 · 재기동 69.7초
- 다른 인스턴스가 열려 있으면 ORA-01126
- cluster_database=FALSE 는 필요 없다

# 스레드별로 쌓이는 아카이브

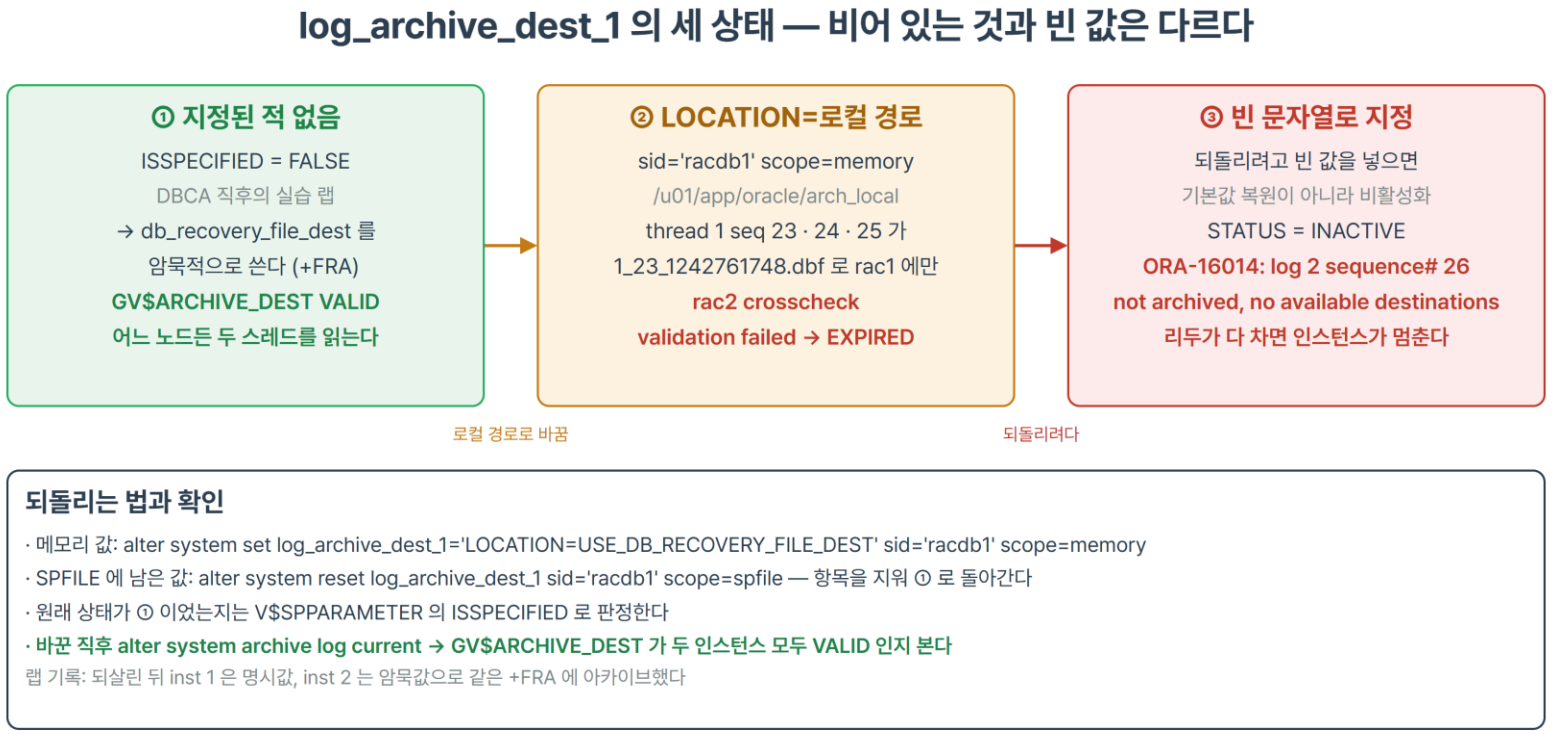
+FRA 는 어느 노드에서든 두 스레드가 보인다




- thread_1_seq_18 · thread_2_seq_13 — 시퀀스는 스레드마다 따로
- %t · %s · %r 셋 모두 RAC 필수 (문서)
- ASM 은 OMF 이름, 형식 파라미터는 파일 시스템 경로에서만

# log_archive_dest_1 의 세 상태

비어 있는 것과 빈 값은 다르다




- ① 지정된 적 없음 — FRA 를 암묵적으로 쓴다
- ② 한 노드 로컬 경로 — rac2 crosscheck 에서 EXPIRED
- ③ 빈 문자열 — INACTIVE, ORA-16014
- 바꾼 직후 archive log current 와 GV$ARCHIVE_DEST 로 검증

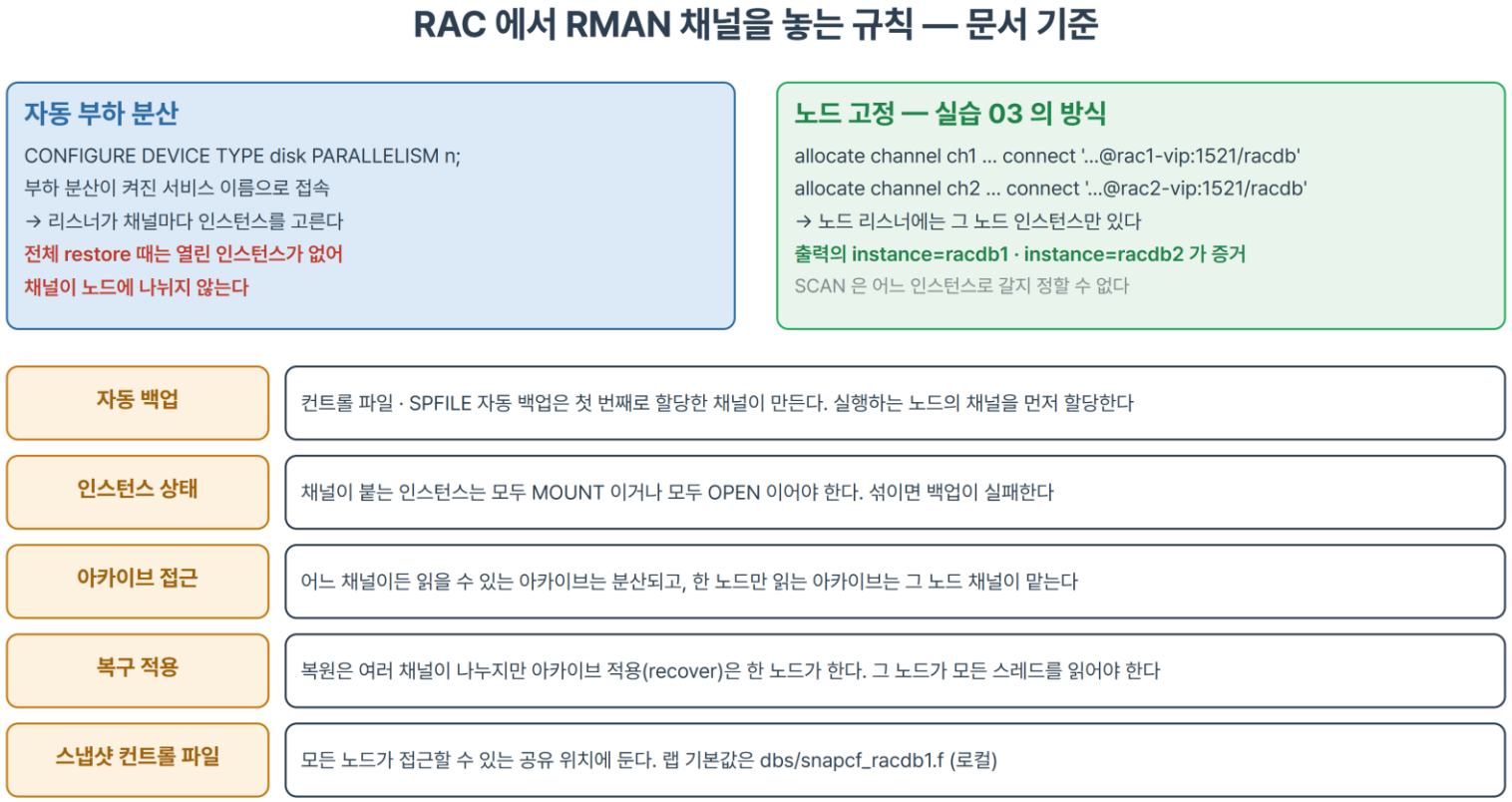
# RAC 에 맞춘 RMAN 구성

show all 에서 챙길 두 줄

- SNAPSHOT CONTROLFILE — 문서: 모든 노드가 접근하는 공유 위치
- 랩 기본값은 dbs/snapcf_racdb1.f, 로컬이다
- CONTROLFILE AUTOBACKUP ON — 카탈로그 없이도 복원
- 자동 백업은 첫 번째 할당 채널이 만든다

# RMAN 채널을 놓는 규칙

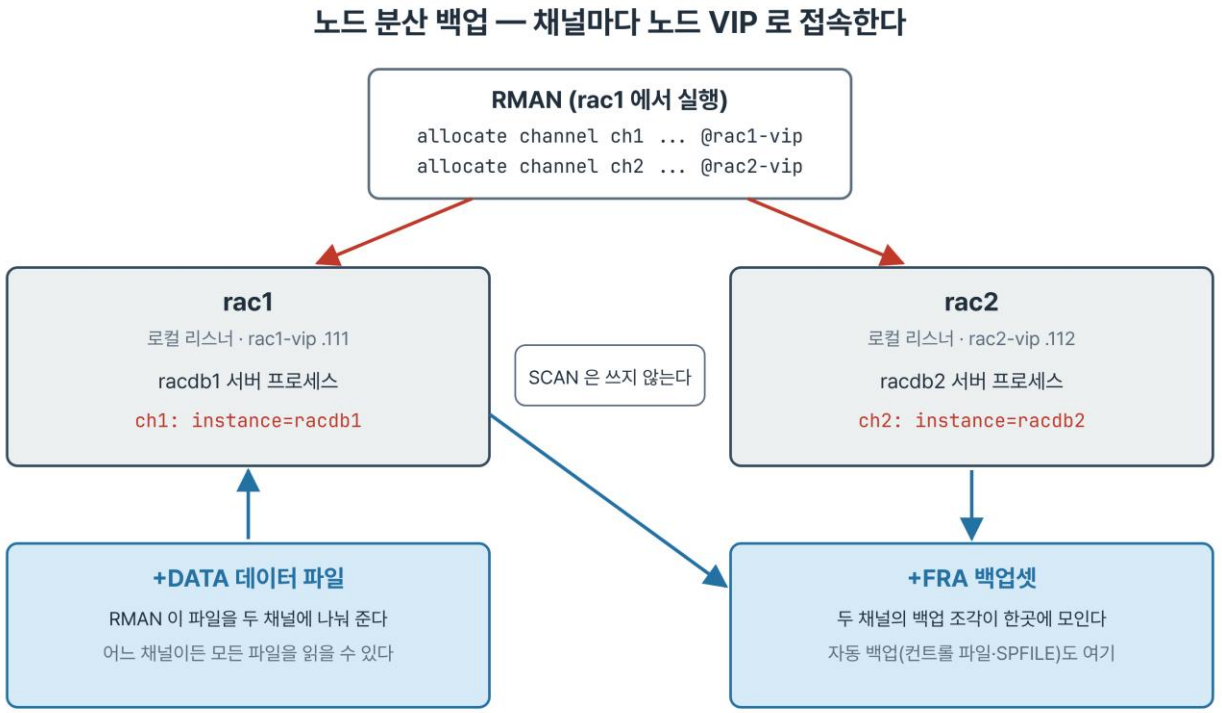
자동 부하 분산과 노드 고정




- 전체 restore 때는 자동 부하 분산이 통하지 않는다
- 채널 인스턴스는 모두 MOUNT 이거나 모두 OPEN
- 아카이브 적용(recover)은 한 노드가 한다

# 노드 분산 백업

채널을 노드 VIP 로 인스턴스에 고정한다




- connect '...@rac1-vip:1521/racdb' · '...@rac2-vip:1521/racdb'
- 증거는 instance=racdb1 · instance=racdb2 두 줄
- 전체 82.0초, 두 채널이 동시에 36초
- 파일 배분은 RMAN 이 정한다

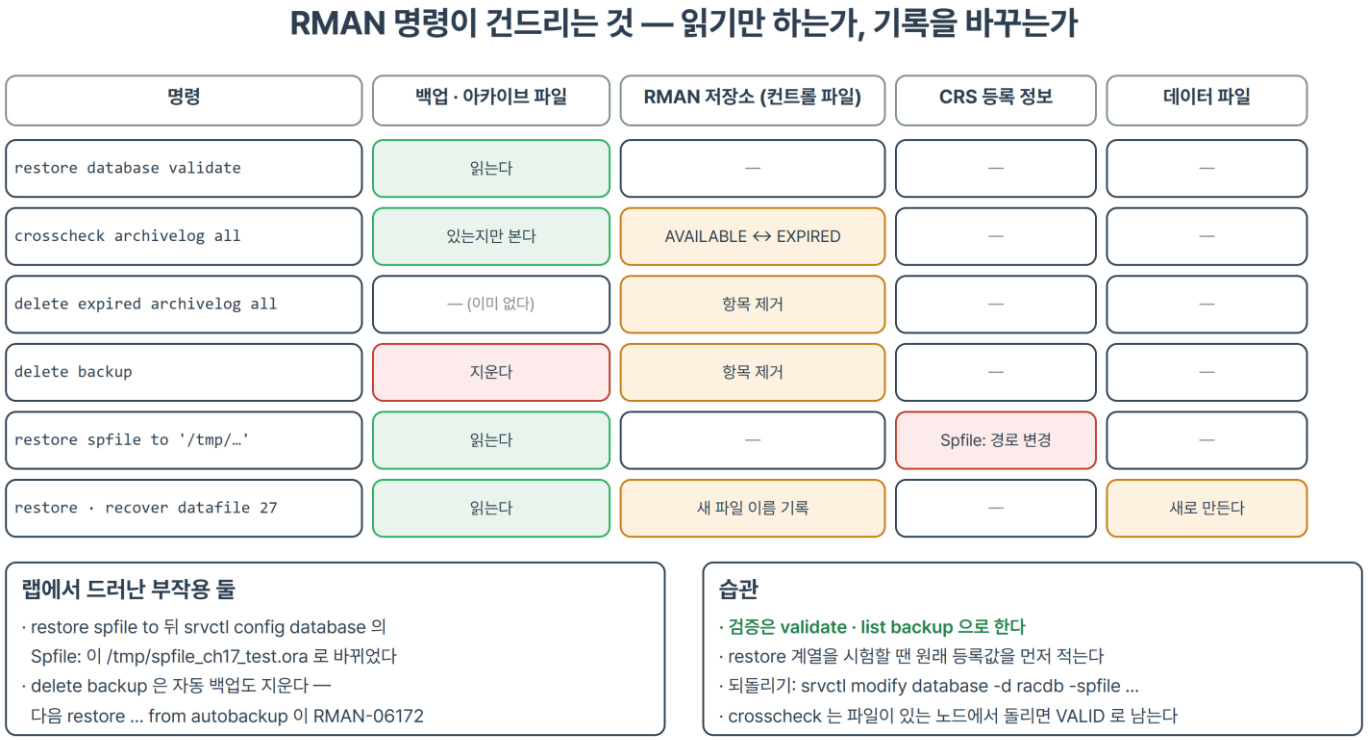
# 백업 검증

어느 노드에서든 쓸 수 있는가

- rac1 restore database validate 71.2초 · rac2 71.4초
- restore archivelog all validate 를 함께
- validate 는 내용을 읽고, crosscheck 는 존재만 본다
- RMAN-06023 — 그 파일의 백업이 없다
- 한쪽만 실패하면 경로 · 아카이브 · 스냅샷 위치 순서로

# RMAN 명령이 건드리는 것

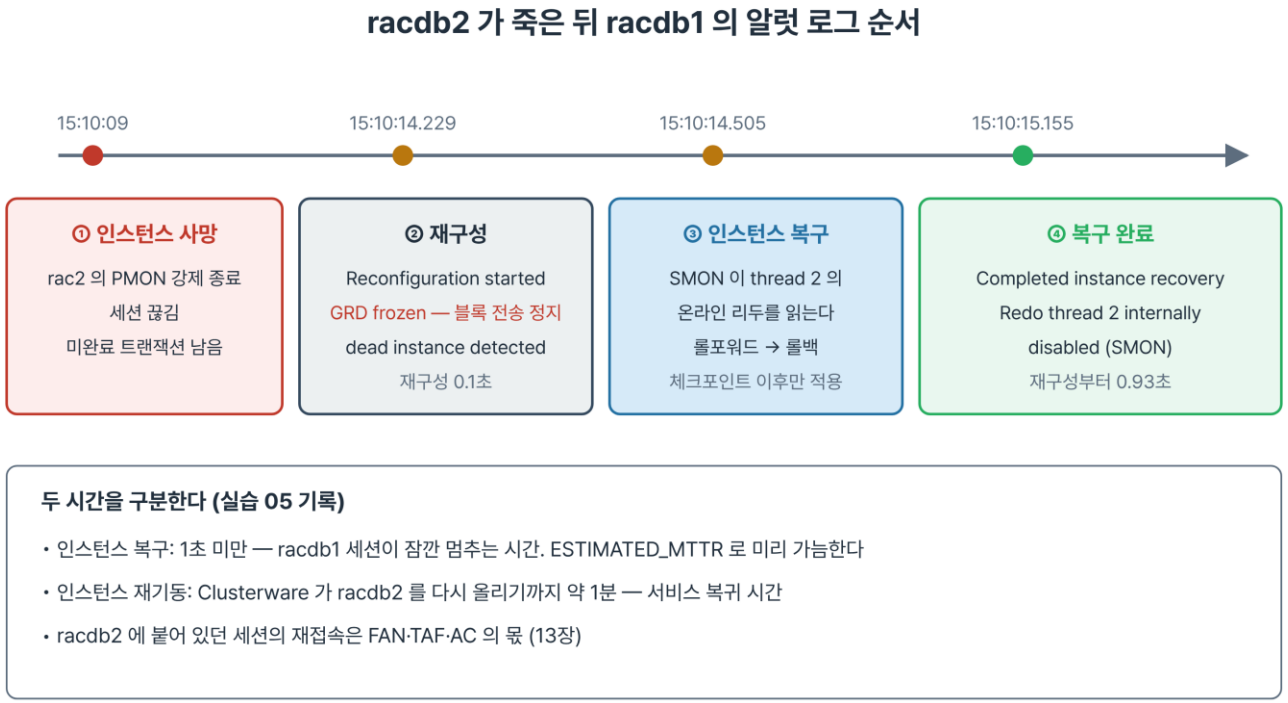
읽기만 하는가, 기록을 바꾸는가




- validate 는 읽기만
- crosscheck · delete expired 는 저장소를 바꾼다
- restore spfile to 는 CRS 의 Spfile: 을 바꾼다
- delete backup 은 자동 백업도 지운다

# 인스턴스 복구

재구성 0.1초, 리두 적용까지 0.93초




- 15:10:09 kill → 15:10:14.229 Reconfiguration started
- GRD frozen — 살아 있는 세션도 잠깐 gc 대기
- SMON 이 thread 2 온라인 리두를 읽는다 (두 멤버)
- ESTIMATED_MTTR 1초 ↔ 실측 0.93초

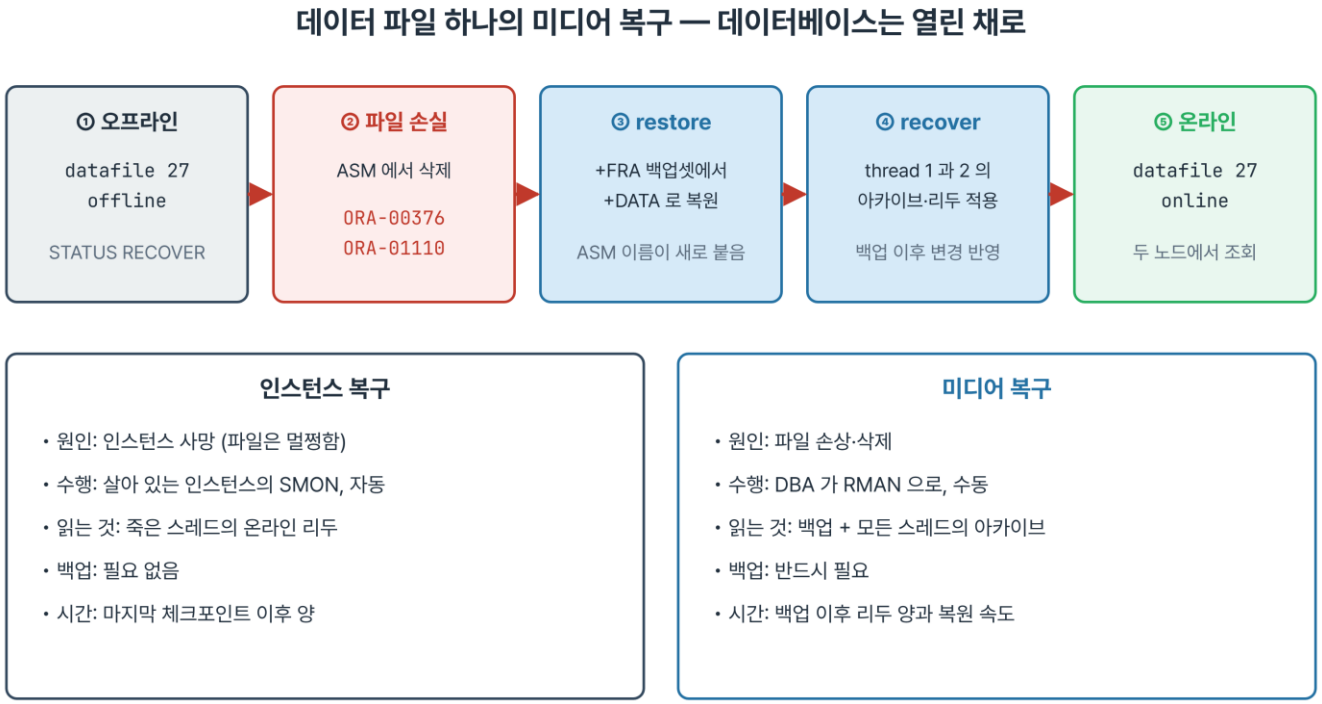
# 복구 시간과 재기동 시간

장애 보고서에서 섞지 않는다

- 인스턴스 복구 0.93초 — racdb1 세션이 잠깐 멈춘다
- 인스턴스 재기동 57초 — Clusterware 가 racdb2 를 올린다
- racdb2 세션은 ORA-03113 으로 끊긴다
- 끊긴 세션을 옮기는 것은 FAN · TAF · AC (4장)

# 미디어 복구

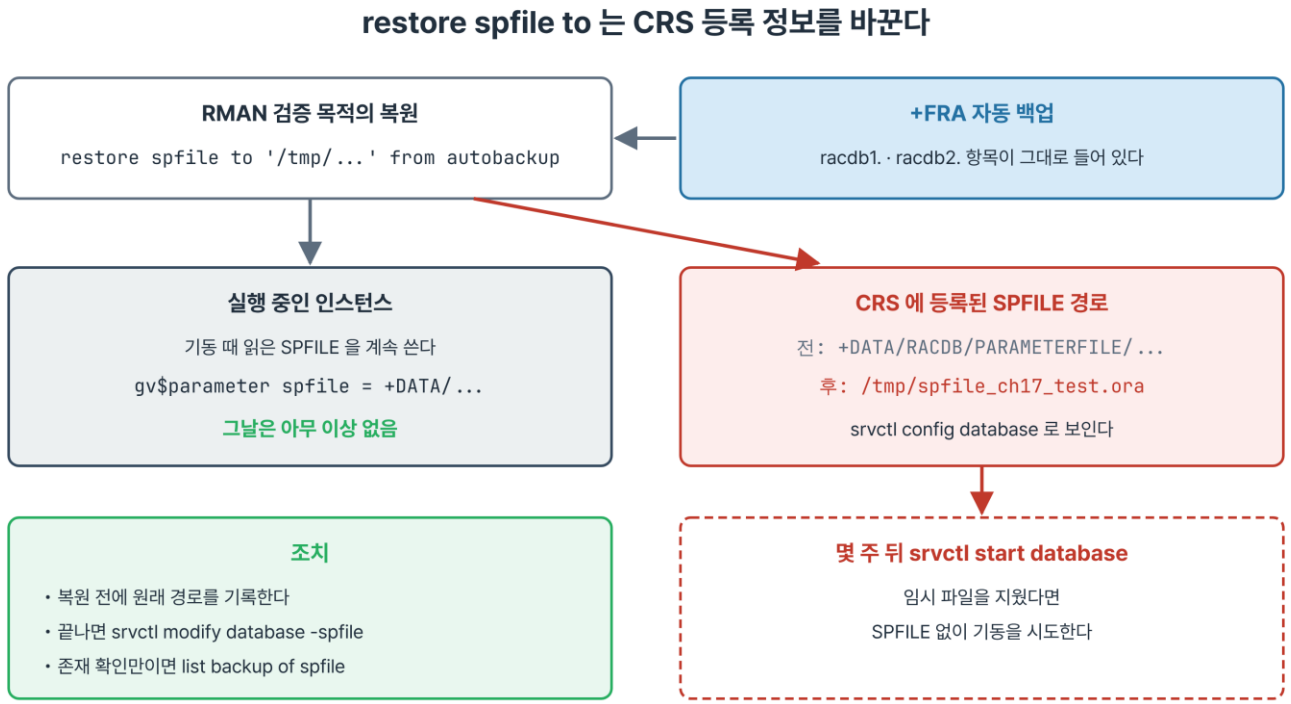
데이터 파일 하나, 데이터베이스는 열린 채




- offline → 파일 삭제 → ORA-00376 · ORA-01110 (두 노드 모두)
- restore · recover · online — 전체 5.3초
- ts_ch17.279.1243091315 → .1243091377
- 복구 노드가 모든 스레드의 아카이브를 읽어야 한다

# restore spfile to 의 부작용

CRS 등록 경로가 바뀐다




- 복원한 SPFILE 에 racdb1. · racdb2. 항목이 그대로
- 실행 뒤 Spfile: /tmp/spfile_ch17_test.ora
- 문서에 없는 동작 — 랩 실측
- srvctl modify database -d racdb -spfile '+DATA/...'

# 컨트롤 파일을 모두 잃었을 때

랩에서 재현하지 않은 절차

- srvctl stop database -o abort, 한 노드에서 startup nomount
- set dbid 1240378513; restore controlfile from autobackup
- mount → recover database (모든 스레드의 아카이브)
- open resetlogs → 나머지 인스턴스 srvctl start

# FRA 와 점검 항목

RAC 에만 있는 두 줄

- FRA 는 공유 위치, DB_RECOVERY_FILE_DEST 는 모든 인스턴스가 같은 값
- 매일: 스레드별 아카이브 시각 · FRA 사용률 · GV$ARCHIVE_DEST VALID
- 주간: 다른 노드 validate · 스레드별 시퀀스 끊김 · 자동 백업
- 변경 후: srvctl config database 의 Spfile: · DBID 기록
- 정리 후 FRA 825MB · 5파일, ARCHIVELOG 는 유지

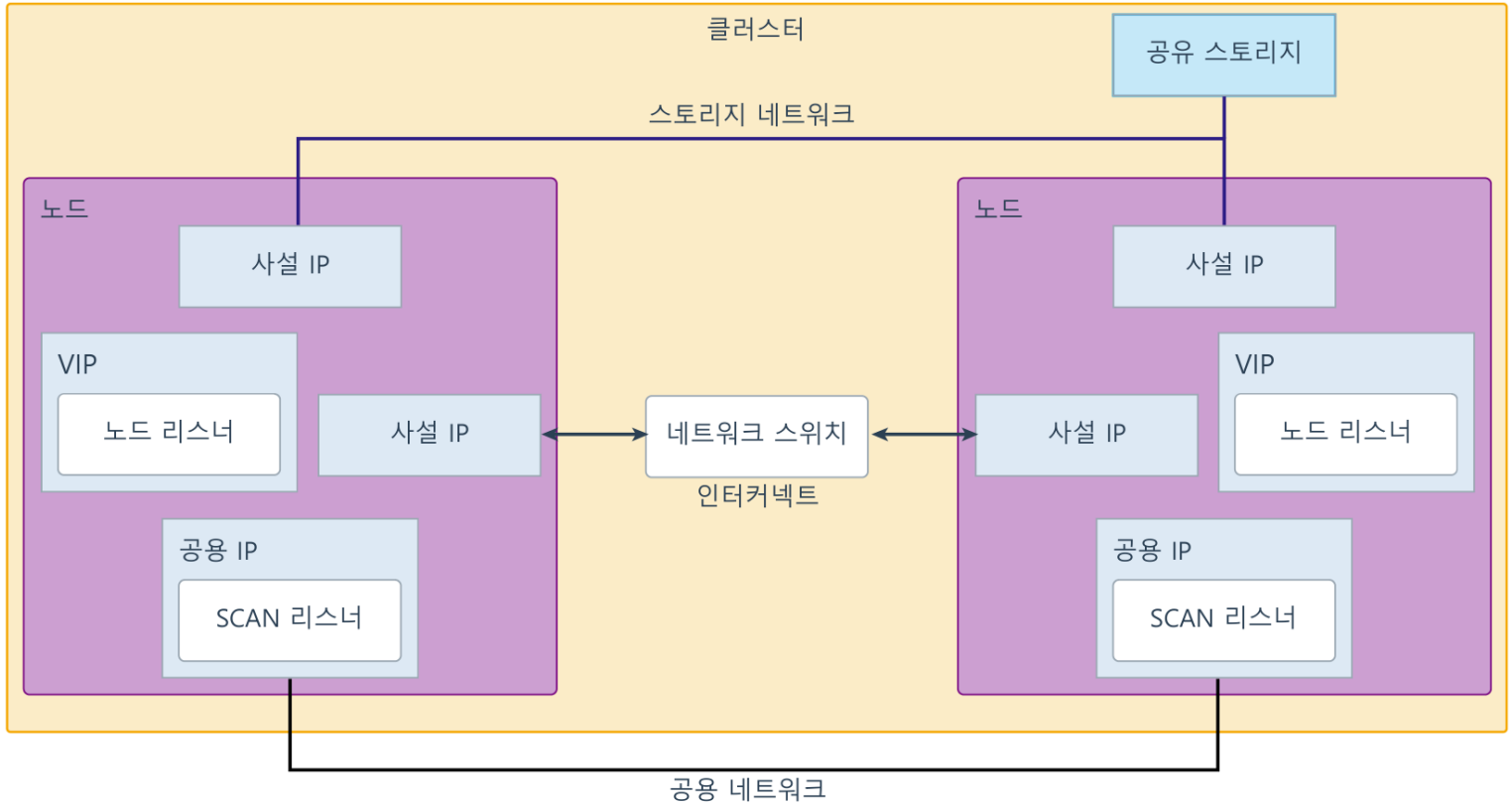
# 6

# 진단 · 성능 · 종합 운영

- 인터커넥트와 HAIP
- gc 대기 이벤트 읽기
- 경합 재현과 원인 객체 특정
- 로그 · TFA · 점검 스크립트
- 자동 재기동의 한도

# 서버 네트워크 구성

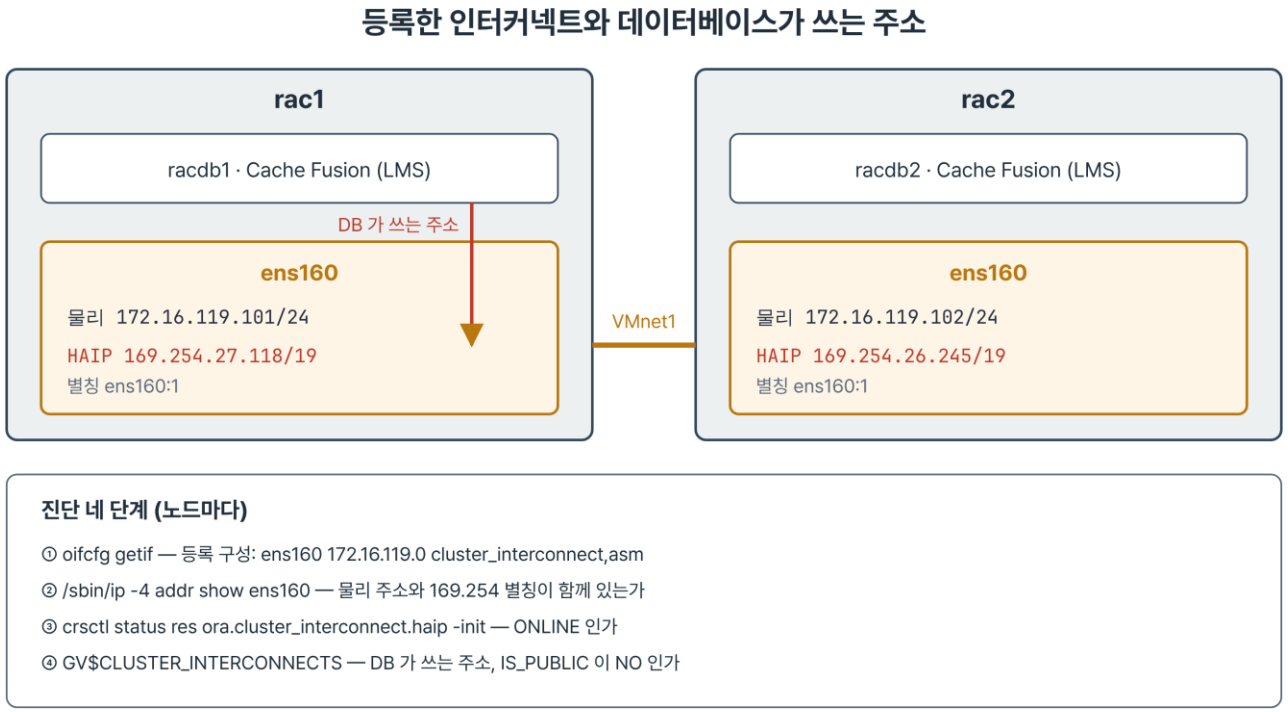
공용 · 사설 · 스토리지 네트워크 (원문 31쪽)




- 공용망: 공용 IP · VIP · SCAN
- 사설망: 인터커넥트 — Cache Fusion 과 하트비트
- 실습: ens160 172.16.119.0 = cluster_interconnect,asm

# 등록한 주소와 쓰는 주소

데이터베이스는 HAIP 별칭 169.254.x.x 를 쓴다




- oifcfg getif: ens160 172.16.119.0
- GV$CLUSTER_INTERCONNECTS: ens160:1 169.254.27.118
- Clusterware 는 물리 대역, DB · ASM · ACFS 는 HAIP
- IS_PUBLIC = NO 인지 꼭 본다

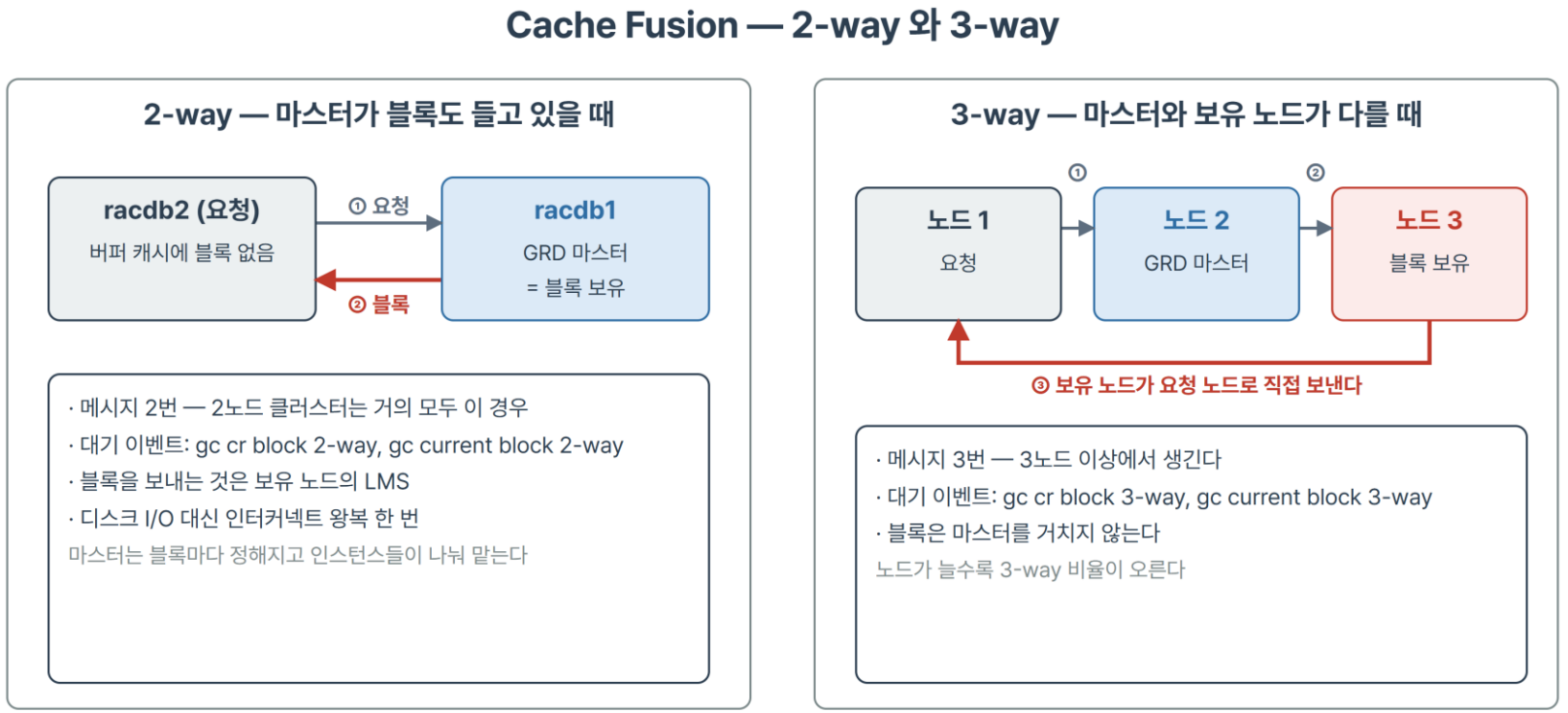
# 인터커넥트 진단 네 단계

ping 만으로는 못 잡는 장애가 있다

- ① oifcfg getif — 무엇을 등록했나
- ② /sbin/ip -4 addr show ens160 — 별칭이 있나
- ③ crsctl status res ora.cluster_interconnect.haip -init — 노드마다
- ④ GV$CLUSTER_INTERCONNECTS — DB 가 무엇을 쓰나

# 2-way 와 3-way

2노드에서는 거의 모두 2-way

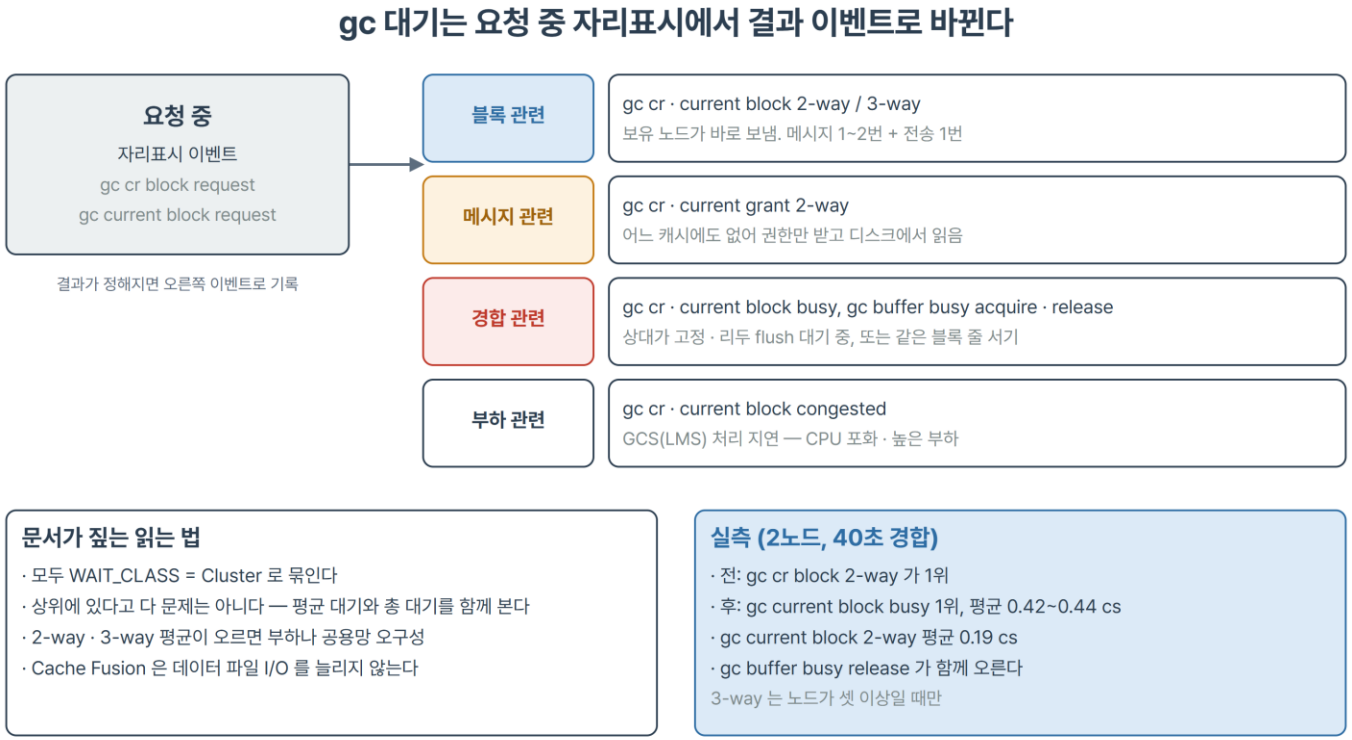



current 블록은 갱신용 최신본(한 시점에 한 인스턴스만 배타 모드), CR 블록은 읽기 일관성용으로 보유 노드가 만들어 보낸 사본

- 2-way: 메시지 1번 + 전송 1번
- 3-way: 마스터가 보유 노드로 넘김, 메시지 2번
- Cache Fusion 은 데이터 파일 I/O 를 늘리지 않는다

# gc 대기의 네 분류

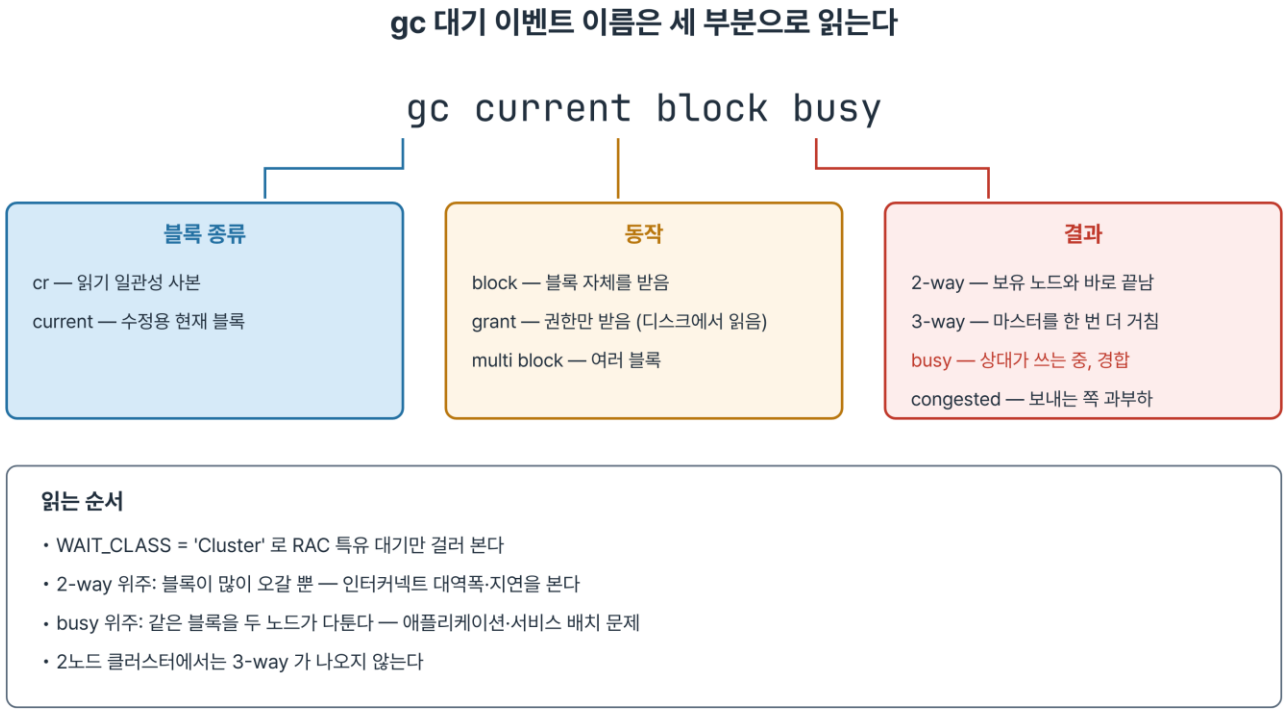
요청 중 자리표시 → 결과 이벤트




- 블록 관련 — 2-way · 3-way, 바로 받음
- 메시지 관련 — grant, 권한만 받고 디스크에서
- 경합 관련 — busy · buffer busy
- 부하 관련 — congested, GCS 처리 지연

# 이름 읽기

블록 종류 · 동작 · 결과




- cr / current — 읽기 일관성 사본인가 수정용인가
- block / grant — 블록을 받았나 권한만 받았나
- busy — 상대가 고정 중이거나 리두를 아직 안 썼다
- buffer busy — 내 노드의 다른 세션 뒤에 줄 섬

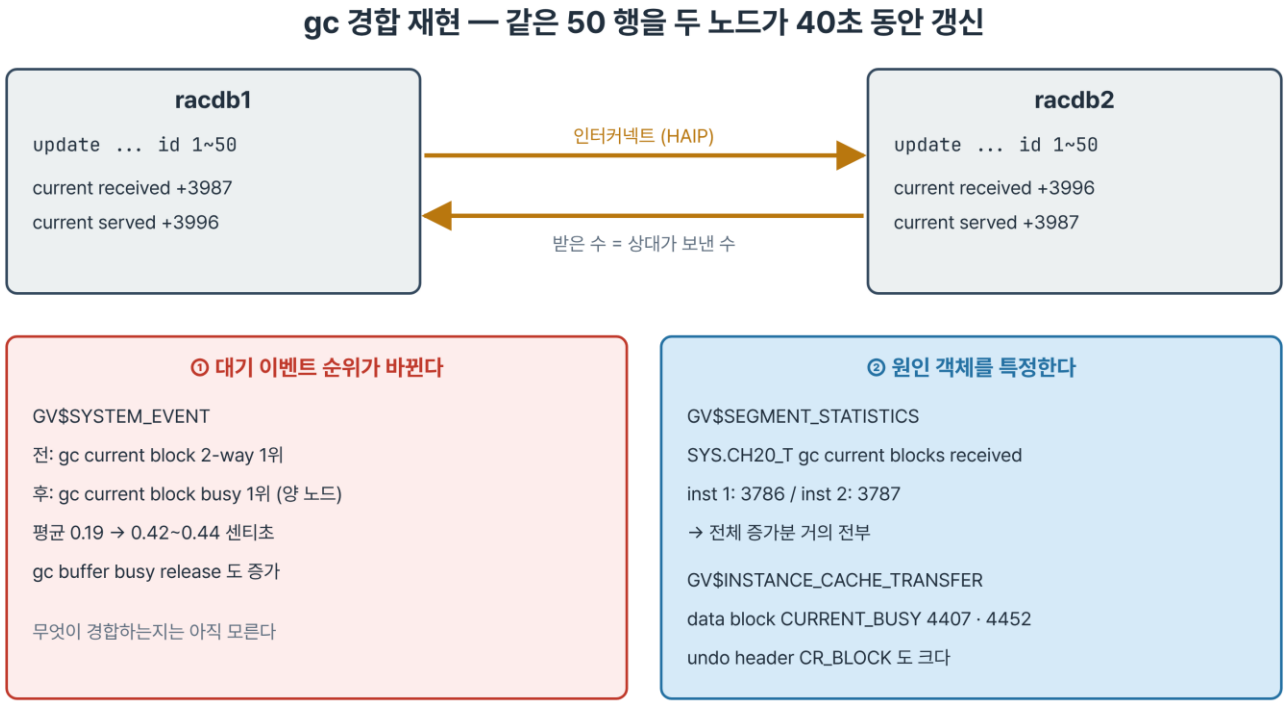
# 경합을 직접 만든다

같은 50행을 두 노드가 40초 동안 갱신

- current received +3987 = 상대 served +3987
- 반대 방향도 +3996 으로 같다 — 40초에 약 8000번
- remote grants 는 +205 · +198 — 블록 자체가 오갔다
- 1위: gc cr block 2-way → gc current block busy
- busy 평균 0.42~0.44 cs, 2-way 0.19 cs

# 원인 객체 특정

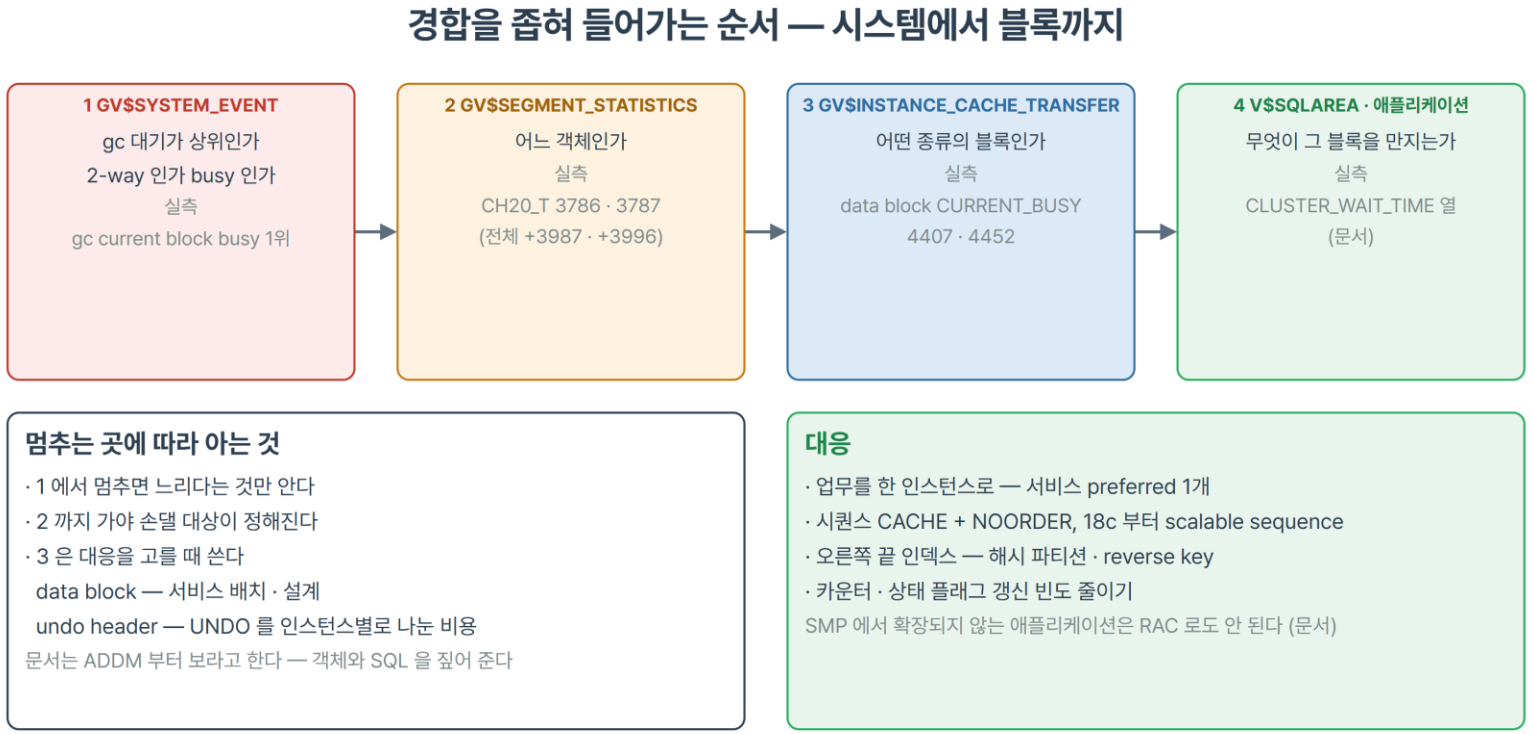
세그먼트 통계가 원인 테이블을 가리킨다




- GV$SEGMENT_STATISTICS: CH20_T 3786 · 3787
- 시스템 전체 증가분의 거의 전부
- OBJECT_NAME 은 대문자 — 소문자면 빈 결과
- GV$INSTANCE_CACHE_TRANSFER: data block CURRENT_BUSY 4407 · 4452

# 좁혀 들어가는 순서

시스템 → 객체 → 블록 종류 → SQL




- V$SQLAREA.CLUSTER_WAIT_TIME — SQL 별 전역 캐시 대기
- 문서의 출발점은 ADDM
- 대응: 서비스로 한 인스턴스에, 시퀀스 CACHE · NOORDER
- 오른쪽 끝 인덱스는 해시 파티션 · reverse key

# AWR · ADDM · ASH 의 RAC 관점

스냅샷은 같은 시점, 저장은 인스턴스별

- AWR — 모든 인스턴스를 같은 시점에, 합계는 저장 안 함
- ADDM — 데이터베이스 · 인스턴스 · Partial
- ASH — Top Cluster Events · Top Remote Instance
- 모든 V$ 에 GV$ 가 있지는 않다 — GV$DIAG_ALERT_EXT 는 ORA-00942

# 로그 지도

어디에 무엇이 남는가




- Clusterware alert.log 를 맨 먼저 — CRS-nnnn
- 데몬 추적 25MB × 10 회전
- 자원 실패 흐름은 crsctl query calog
- srvctl · CVU 는 SRVM_TRACE=TRUE

# TFA 로 모으기

19.3 홈에는 diagcollection.pl 이 없다

- tfactl diagcollect -crs -last 1h -node local -tag ch20
- 289초 · 63MB — 범위를 안 주면 훨씬 크다
- 시간 단위는 h · d 만 (5m 거부)
- SR 은 -srdc dbrac_evictions -node all
- 저장소 430MB — 정기적으로 지운다

# 종합 점검 스크립트

계층 순서 = 진단 순서

- 클러스터 · 알림 · 서비스 · 인스턴스 · 스토리지 · 백업
- FAIL 과 WARN 을 나눈다, 종료 코드 = 실패 건수
- 기준선 17 · 6 을 숫자로 박는다
- 11.5초 — 매일 돌린다
- 있어야 할 것의 목록(V$THREAD)으로 조회한다

# 장애를 내고 점검한다

racdb2 PMON kill 20초 뒤




- 인스턴스 계층만 FAIL, 클러스터 계층에 ora.racdb.db WARN
- 스토리지 · 백업 행은 조용히 사라짐 — GV$ 는 산 인스턴스만
- 노드 VIP 로는 ORA-12514, SCAN 으로는 racdb1
- 약 101초 뒤 CRS 가 자동 재기동, 실패 0건

# 자동 재기동의 한도

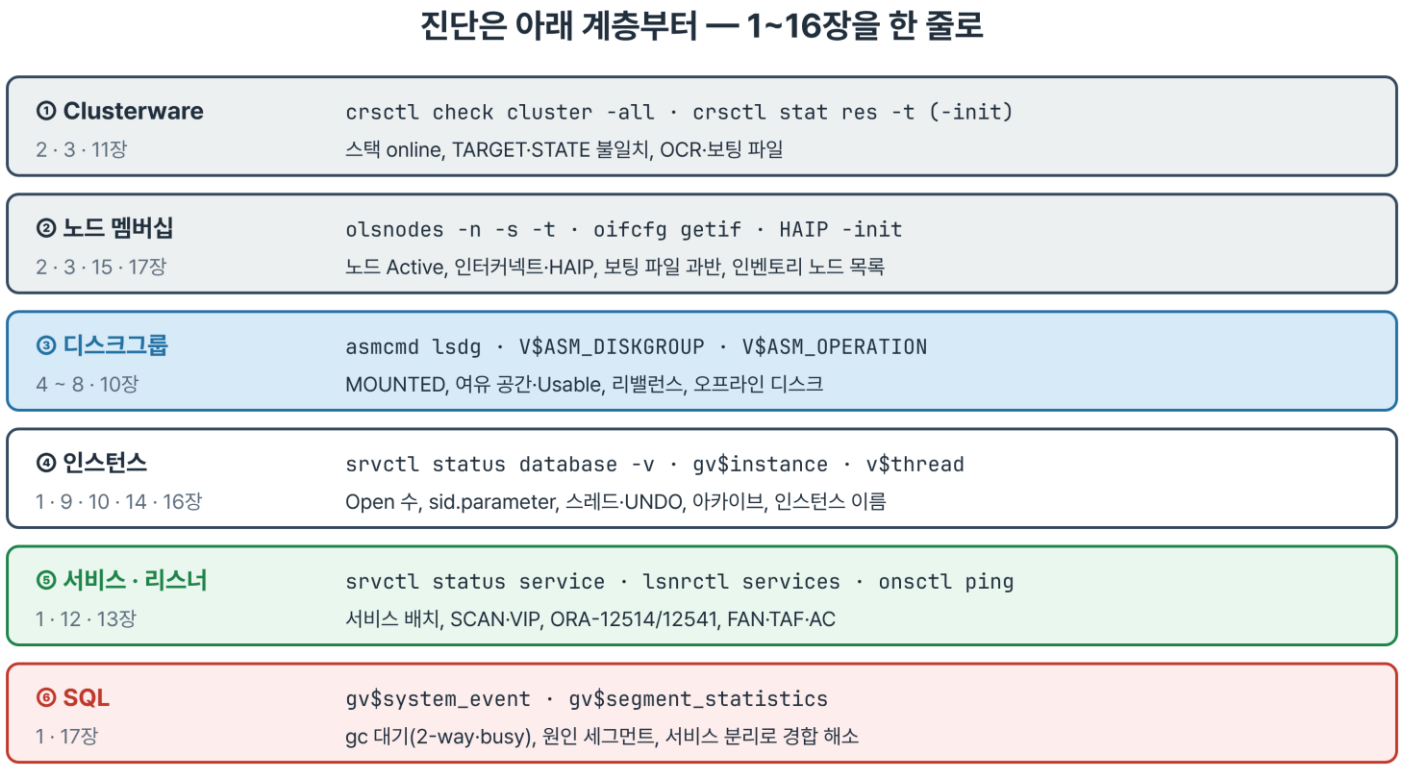
RESTART_ATTEMPTS=2 · UPTIME_THRESHOLD=1h




- 4분 간격 두 번째 장애 → CRS-2771 · CRS-2769
- TARGET 은 ONLINE, STATE 만 OFFLINE
- 사람이 srvctl start instance (42.9초)
- 한 시간에 두 번 = 원인이 남아 있다

# 진단 순서

아래 계층부터 올라간다




- Clusterware → 노드 멤버십 → 디스크그룹
- → 인스턴스 → 서비스 · 리스너 → SQL
- RAC 와 Data Guard 는 서로를 대신하지 않는다

# 요약

- 리두 · UNDO 만 인스턴스별, 나머지는 공유 — 파일은 모두 공유 디스크그룹에
- ora. 자원은 srvctl, 스택과 자원 모델은 crsctl. OFFLINE 은 TARGET 부터
- 서비스는 OCR · 인스턴스 · 리스너 세 곳에 있다. relocate 는 새 접속만 옮긴다
- FAN 이 먼저 알리고, TAF 는 다시 붙고, AC 는 요청을 재생한다
- 스레드마다 아카이브, 채널은 VIP 로 노드에 고정, 검증은 다른 노드에서
- gc 대기는 블록 · 메시지 · 경합 · 부하로 읽고, 자동 재기동에는 한도가 있다

# 퀴즈

- 1. rac1 에서 THREAD 절 없이 리두 그룹을 추가하면 어느 스레드에 붙는가?
- 2. crsctl stop crs 뒤 다시 올렸는데 racdb 가 내려가 있다. 왜인가?
- 3. relocate 직후 서비스 이름으로 rac1:1521 에 붙으면 무엇이 나오는가?
- 4. 서비스는 TRANSACTION, 클라이언트는 TYPE=SELECT 면 FAILOVER_TYPE 은?
- 5. 로컬 경로에 쌓인 아카이브는 다른 노드의 crosscheck 에서 어떻게 보이는가?
- 6. 한 시간 안에 두 번 죽은 racdb2 를 CRS 가 다시 올리지 않은 이유는?

# 퀴즈 정답

- 1. 접속한 인스턴스(racdb1)의 스레드 1
- 2. srvctl 로 먼저 내린 DB 는 AUTO_START=restore 라 내려간 채 남는다
- 3. ORA-12514 — 노드 리스너가 그 서비스를 더는 모른다. 접속은 SCAN 으로
- 4. TRANSACTIONAL — 서비스 설정이 클라이언트보다 우선한다
- 5. EXPIRED. 그 시퀀스에서 복구가 멈춘다
- 6. RESTART_ATTEMPTS 를 넘었고 UPTIME_THRESHOLD 전이라 초기화되지 않았다 (CRS-2771)

# 실습과 자료

- 실습 트랜스크립트 — RAC/5일과정/아키텍처/RAC_ADMIN실습 (13장 · 14장 · 15장 · 16장 · 17장 · 20장 실습 54개)
- 5일 과정 사본 — RAC/5일과정/10장~14장 · 17장
- 공식 문서 — RAC Administration and Deployment Guide 19c, Clusterware Administration Guide 19c (docx 부록 D)
- 본문 절과 실습 대응 — docx 부록 C, 오류 코드 — 부록 A, 실습 환경 값 — 부록 B